

Master's thesis

Master's Programme in Computer Science

# Performance, Safety, and Engineering Trade-offs of Rust and C++ in Nokia RAN Control-Plane Systems

Ha Vien

September 29, 2026

Faculty of Science

University of Helsinki

**Contact information**

P. O. Box 68 (Pietari Kalmin katu 5)
00014 University of Helsinki,Finland

Email address: info@cs.helsinki.fi
URL: http://www.cs.helsinki.fi/

HELSINGIN YLIOPISTO – HELSINGFORS UNIVERSITET – UNIVERSITY OF HELSINKI

| Tiedekunta — Fakultet — Faculty | Koulutusohjelma — Utbildningsprogram — Study programme |
|---|---|
| Faculty of Science | Master's Programme in Computer Science |

Tekijä — Författare — Author

Ha Vien

Työn nimi — Arbetets titel — Title

Performance, Safety, and Engineering Trade-offs of Rust and C++ in Nokia RAN Control-Plane Systems

Ohjaajat — Handledare — Supervisors

Prof. Keijo Heljanko, Christoph Luneburg, Petteri Kotimaa

| Työn laji — Arbetets art — Level | Aika — Datum — Month and year | Sivumäärä — Sidoantal — Number of pages |
|---|---|---|
| Master's thesis | September 29, 2026 | 44 pages, 8 appendix pages |

Tiivistelmä — Referat — Abstract

Radio access network (RAN) control planes are written mostly in C++ for performance, at a well-known cost in memory safety and concurrency. Rust promises comparable performance with compile-time safety, but its fit for this latency-critical setting is unproven.

The thesis compares modern C++ and Rust on a user equipment (UE) Context Management service that keeps per-device state, as a 5G base-station control plane (gNB-CU-CP) does. Average-throughput comparisons leave four dimensions open: tail latency, the defect classes Rust prevents at compile time, safety-model overhead, and incremental adoption.

One language-agnostic specification is frozen, then implemented in C++ and Rust with matched algorithms, concurrency model, allocator, and build settings, with forced-idiom gaps logged. A synthetic workload is increased to overload, both versions pass the same parity tests, and the evaluation adds fault injection and a foreign function interface (FFI) prototype.

Safe Rust rejects all seven injected memory-safety and data-race classes at compile time. The same seven build in C++. Seven further classes fail in both. Below saturation the paired test resolves 1.5 $\mu$s, and an end-to-end call spends only 0.250% on language work. Both languages meet the offered rate through 392 kops/s. With the fill inside the timed window, C++ holds 1.7× the throughput at 784 kops/s. On this same host, moving the fill to after the window removes that large gap and leaves a smaller Rust lead. Overload service latency stays unresolved.

**ACM Computing Classification System (CCS)**
Software and its engineering → Software notations and tools → General programming languages
Computer systems organization → Real-time systems → Real-time system specification
Networks → Network types → Mobile networks

Avainsanat — Nyckelord — Keywords

Rust, C++, RAN control plane, tail latency, memory safety, fault injection, incremental adoption

Säilytyspaikka — Förvaringsställe — Where deposited

Helsinki University Library

Muita tietoja — övriga uppgifter — Additional information

Software study track

# Contents

**List of Figures** iii

**List of Tables** iv

**List of acronyms** v

**1 Introduction** 1

1.1 Motivation . . . 2
1.2 Industrial context . . . 3
1.3 Problem statement . . . 4
1.4 Research questions and hypotheses . . . 5

**2 Background: Systems Languages for Latency-Critical Software** 7

2.1 RAN control-plane systems and the gNB-CU-CP . . . 7
2.2 Research gap . . . 8

**3 Research Methodology** 11

3.1 The UE Context Management subsystem . . . 11
3.2 Research approach . . . 12
3.3 Methods per research question . . . 13
3.4 Fairness as a controlled variable . . . 14
3.5 Statistical treatment . . . 14
3.6 Reproducibility and research artefacts . . . 15

**4 Design and Implementation** 16

**5 Experimental Setup and Evaluation** 18

5.1 Testbed setup . . . 18
5.2 Workload and measurements . . . 19
5.3 Results . . . 20
5.3.1 Below saturation (RQ1, RQ3) . . . 20
5.3.2 Past saturation (RQ1) . . . 22

5.3.3 Resource use (RQ3) . . . 24
5.3.4 Safety via fault injection (RQ2) . . . 25
5.3.5 Migration (RQ4) . . . 27

**6 Discussion** **28**
6.1 Summary of findings . . . 28
6.2 Discussion on the research questions . . . 28
6.3 Revisiting the hypotheses . . . 31
6.4 Engineering guidelines for incremental adoption . . . 31
6.5 Where this study sits . . . 32
6.6 Ethical and professional reflection . . . 32

**7 Conclusion and Future Work** **33**

**Statement on the Use of AI Tools** **35**

**Bibliography** **36**

**A Key terms**

**B Host and campaign configuration**

# List of Figures


2.1 Where the studied subsystem sits in the 5G NG-RAN . . . . . 8
2.2 A checklist of earlier comparisons . . . . . 10

3.1 UE-context lifecycle state machine . . . . . 11

4.1 Shared-nothing, thread-per-shard architecture . . . . . 16

5.1 The two test workloads . . . . . 19
5.2 The load sweep: tail latency and achieved throughput . . . . . 21
5.3 Where the end-to-end clock goes . . . . . 22
5.4 How small the language's share of a round-trip is . . . . . 22
5.5 Four load points, from a light load to overload . . . . . 23
5.6 Campaign and deferred-fill throughput on this host . . . . . 24
5.7 Peak memory and CPU of the four builds . . . . . 25
5.8 Where the guarantee stops: the outcome matrix . . . . . 26
5.9 The thin layer where C++ calls the Rust service . . . . . 27

6.1 Where Rust's compile-time guarantee stops . . . . . 30

# List of Tables

3.1 How each research question is answered . . . . . 13

5.1 Per-operation latency at the representative load . . . . . 20

A.1 Key terms . . . . . i

B.1 The three measurement hosts . . . . . ii
B.2 Configuration of the service, the workload, and the campaign . . . . . v
B.3 How much the delay changes from one call to the next . . . . . vi
B.4 Where the process's extra memory sits . . . . . vi

# List of acronyms

Each acronym below is also written out where the text first uses it. Terms that are not acronyms, such as *tail latency* and *shard*, are collected in Appendix A.

| Acronym | Expansion |
|---|---|
| 3GPP | 3rd Generation Partnership Project, the body that specifies the 5G standards |
| ABI | Application binary interface |
| AI | Artificial intelligence |
| AI-RAN | An architecture in which AI and radio access network workloads share one accelerated infrastructure |
| API | Application programming interface |
| ASN.1 | Abstract Syntax Notation One |
| CPU | Central processing unit |
| CVE | Common Vulnerabilities and Exposures |
| CWE | Common Weakness Enumeration |
| DPDK | Data Plane Development Kit |
| EU | European Union |
| FFI | Foreign function interface |
| FIFO | First in, first out |
| FNV-1a | Fowler–Noll–Vo hash function, variant 1a |
| GCC | GNU Compiler Collection |
| gNB | Next-generation NodeB, the 5G base station |
| gNB-CU | gNB central unit |
| gNB-CU-CP | gNB central unit control plane, the domain this work sits in |
| gNB-CU-UP | gNB central unit user plane |
| gNB-DU | gNB distributed unit |
| HDR | High dynamic range, of the latency histogram |
| IMT-2030 | International Mobile Telecommunications-2030, the ITU framework for 6G |
| ITU | International Telecommunication Union |
| KPI | Key performance indicator |
| kops/s | Kilo-operations per second |
| KVM | Kernel-based Virtual Machine |
| MPSC | Multi-producer, single-consumer |
| NG-RAN | Next-generation radio access network |
| PHY | Physical layer |
| RAII | Resource Acquisition Is Initialization |
| RAN | Radio access network |
| RQ | Research question |
| RRC | Radio Resource Control |
| RSS | Resident set size |
| UE | User equipment, an end device such as a phone or a modem |
| UECM | UE Context Management, the subsystem reimplemented and measured here |

# 1 Introduction

A mobile network is the system that connects a phone to the rest of the network. For a long time, the radio work in that system ran inside a box built for this job alone. One vendor sold the box and the software together. That arrangement is called vertically integrated equipment. The same work is now a program on an ordinary server, a general-purpose machine. The radio access network (RAN) is this radio part. It was the piece most tied to those special boxes, and it is part of the same change. The stack is the set of layers from the radio signal up to the core network.

In 5G, the base station, formally the next-generation NodeB (gNB), is split into a central unit and one or more distributed units. The central unit is split again into control-plane and user-plane functions [1, 2]. Open-RAN pushes the same idea further: the RAN becomes virtualised, multi-vendor software linked by open interfaces [3]. The control plane that results, the gNB central unit control plane (gNB-CU-CP), is a large, stateful, highly concurrent program. It tracks every connected device. The standards call each device a user equipment (UE). The control plane runs Radio Resource Control (RRC) signalling [4] and coordinates mobility and handovers, all within strict latency limits.

A single site can run thousands of connection-management and mobility procedures per second. Each procedure has to complete within the millisecond-scale budget the 5G system sets for control-plane signalling [5]. A deployed control plane may track millions of active devices at once. As the RAN moves to cloud-native, disaggregated deployments, the correctness, reliability, and timing of this software increasingly set the quality of the whole network. An outage here is not a local defect. Neither is a tail-latency spike: a few calls take much longer than the rest, even when the average still looks fine. Every device the site serves is affected [6, 7].

This control-plane software is, today, written mostly in C++. The choice is a practical one: C++ offers the low-level control, mature tooling, and performance that latency-critical systems demand. But that performance comes with a well-known cost. C++ places the burden of memory and concurrency management on the programmer, and decades of experience show that this burden is routinely mishandled, producing use-after-free errors, double-frees, data races, and related defects that are expensive to find, dangerous in production, and responsible for a large share of security vulnerabilities in industry practice [8, 9]. Rust has emerged as the first mainstream systems language to promise an escape from this trade-off: memory safety and data-race freedom enforced at compile

time on machine-checked foundations [10, 11], with performance intended to be competitive with C++ [12]. Whether that promise holds for a telecom-grade, latency-critical control plane, where predictability under overload matters as much as raw speed, is not well understood. This thesis, carried out at Nokia, addresses the question directly: it develops two functionally equivalent implementations of a representative RAN subsystem, a UE Context Management service that keeps per-device state, as a gNB-CU-CP does, in modern C++ and in Rust, and compares them under a realistic synthetic signalling workload.

Three terms recur throughout. The *gNB-CU-CP* is the control-plane class this work sits in. *UE Context Management (UECM)* is the compact service reimplemented and measured. *Tail latency*, the p99, p99.9 and p99.99 percentiles rather than the mean, is the property that, more than average speed, decides whether such a system meets its latency budgets [6, 7]. Every acronym is written out where the text first uses it, and the List of acronyms in the front matter collects them. The terms that are not acronyms are fixed in Appendix A for readers outside telecom.

## 1.1 Motivation

Three trends make this comparison timely. The first is the cloud-native evolution of the RAN. Control-plane functions now run on general-purpose servers, so the software is longer-lived, more concurrent, and must meet its latency budgets without the determinism of purpose-built hardware. Open-RAN disaggregation turns the base station into virtualised, multi-vendor software [3]. A latency regression or a memory-corruption defect is then a network-wide risk, and the language choice is an engineering decision.

The second trend is artificial intelligence (AI). AI used to be only a tool. It helped choose radio settings, such as how much power to send. That tool did not have to run on the radio's own server. Now AI programs also run on that server, next to the radio software [13]. Both run at the same time, so they share one processor. International Mobile Telecommunications-2030 (IMT-2030) is the 6G framework of the International Telecommunication Union (ITU). It lists AI together with communication as one named job for 6G [14]. The architecture forming around that job is AI-RAN, in which AI and radio work share one accelerated server [15]. A control plane there no longer has the host to itself. It shares the central processing unit (CPU), memory, and cache with inference work it does not schedule. When that neighbour takes capacity, the control plane saturates sooner: it stops keeping up with demand at a lower load than it would alone. Chapter 5 finds the

widest campaign-clock gaps past that point. Later diagnostic sweeps do not reproduce either gap, so they cannot be read as a ranking of the two services.

The third is the shift toward memory-safe languages. Large software organisations report that the majority of their serious security vulnerabilities stem from memory-safety errors, and that adopting memory-safe languages reduces them [8, 9, 16, 17]. Independent work argues that memory safety now needs standardised requirements if it is to be procured at all [18]. Cybersecurity agencies urge a move away from C and C++, and the European Union (EU) Cyber Resilience Act adds legal duties from September 2026 and December 2027 [19, 20, 21]. The Act names no programming language. Direct evidence for a RAN control plane is still missing. Providing it is the motivation for this work.

## 1.2 Industrial context

This thesis is carried out at Nokia. The company is one of the small number of vendors that build, maintain, and evolve production RAN software end to end. The five largest suppliers took 96% of worldwide RAN revenue over the first three quarters of 2025, with concentration by Herfindahl–Hirschman Index at a ten-year high [22, 23]. The subsystem it studies, UECM, is a compact model of the per-UE state work a gNB-CU-CP does.

In the architecture of the 3rd Generation Partnership Project (3GPP), the gNB-CU-CP is the control-plane half of the central unit. It hosts RRC and manages the UE context lifecycle [1, 4], under the millisecond-scale budgets the 5G system sets for control signalling [5]. Chapter 2 places that control plane in the split 5G base station, the gNB. Because it is written in C++, it inherits that language's performance and its memory-safety and concurrency hazards. The three trends of Section 1.1 meet here. Software of this class runs on live operator networks. The language choice is a production concern.

UECM is small enough to reimplement faithfully and reason about in full. It still carries the properties that make control-plane code hard. The implementation follows a public, frozen specification, not proprietary code, so readers outside Nokia can reproduce the comparison.

**A decision with a generational horizon.** A language choice in software of this class outlives the generation it was made for. 3GPP has allocated Release 20 to the 6G studies and Release 21 to the normative work. The ITU asks for technology proposals in early 2029 and for specification submissions by mid-2030 [24]. That work covers migration from 5G and interworking with it. A control plane written now is maintained alongside the next

generation, not retired. IMT-2030 still names security and resilience as a design principle and still carries the ultra-reliable low-latency scenario in strengthened form [14].

The measurements here use a current-generation subsystem and published requirements. No claim is made about requirements that have not been written.

**A concrete failure mode.** A moving train full of passengers stands for the connected UEs at one site. A handover moves one passenger from one cell to another, and the call can continue. Each passenger has a stored record, the UE context. When many passengers move at once, the control plane is under a mobility storm. One passenger leaves, so that record is deleted, while a message for that passenger is still being handled. Which of those two steps finishes first is a race condition. If the read comes after the delete, it is a use-after-free. In C++ this still compiles. Tests that do not hit that ordering can pass. When many calls run at once, the bug can corrupt another passenger's record. That race can crash the thread that is serving the call. A few calls can become much slower than the rest. That jump in the slowest calls is a tail-latency spike. In safe Rust the same design does not compile: ownership and borrowing reject the read of a record that has already been deleted. That is why this thesis measures both *safety* (which defects are prevented, and when) and *predictability* (tail latency under overload), not average throughput alone.

## 1.3 Problem statement

Two properties of this domain make the language choice consequential. First, manual memory and concurrency management in C++ exposes the software to defects such as use-after-free, double-free, data races, and iterator invalidation. These defects are silent at compile time and costly to find and fix at telecom scale. Second, what governs whether RAN latency budgets are met is not average throughput but worst-case and tail behaviour under load and overload [6], a property that is easy to regress and hard to measure correctly. Rust promises to remove the first class of defect at compile time while claiming performance competitive on the second, but that claim rests mostly on general-purpose benchmarks rather than on stateful, concurrent, latency-critical telecom workloads.

Bringing these threads together, the concrete problem this thesis addresses is the following. Telecom vendors are under growing pressure, both technical and increasingly regulatory, to adopt memory-safe languages, yet for the RAN control plane they lack *direct, controlled* evidence on which to base that decision. It is not established whether a Rust implementation of a stateful, concurrent control-plane subsystem can match modern C++ in *tail*

*latency and predictability under overload* (as opposed to average throughput). It is also unknow whether Rust's compile-time guarantees actually eliminate the specific defect classes that affects the existing C++ code, and what incremental adoption *costs* inside a large C++ codebase. The available evidence is largely general-purpose and average-case [12, 25], leaving these three questions (predictability, prevented-defect scope, and migration cost) open for a representative subsystem. Answering them under controlled, reproducible conditions is the problem this thesis sets out to solve.

## 1.4 Research questions and hypotheses

The thesis is organised around four research questions. They put latency predictability first, then defect scope, then resource cost, then incremental adoption:

**RQ1** How do Rust and modern C++ compare in latency predictability (median, p99, p99.9 and p99.99 tail latency, and the maximum) for a RAN control-plane workload under increasing load and overload?

**RQ2** Which classes of memory-safety and concurrency defects that occur in C++ are eliminated at compile time in the Rust implementation, and at what stage and cost are the remaining ones detected?

**RQ3** What is the overhead, if any, of Rust's safety model (bounds checks, ownership-driven copies, abstraction cost) on throughput, CPU utilisation, and memory behaviour?

**RQ4** What are the practical implications (effort, foreign function interface (FFI) interoperability, and risk) of adopting Rust incrementally within an existing C++ RAN software ecosystem?

Three hypotheses are written down before any measurement.

**H1** For this workload the two languages are performance-equivalent below saturation, with any difference only in the far tail.

**H2** Safe Rust prevents at compile time the memory-safety and concurrency defect classes that C++ only tries to catch after the program is built.

**H3** Calling the Rust service from C++ through the FFI adds only a small share of one signalling call's time. The real cost of adding Rust a piece at a time is engineering work, not slower calls.

Chapter 6 revisits all three hypotheses against the facts.

The RQ2 list has fourteen defect classes. Safe Rust is expected to stop seven of them at compile time. It is expected not to stop the other seven. That split is the boundary the study looks for, not only a check that the guarantee works. The seven boundary classes are lock-order deadlock, lost update, reference-cycle leak, release-build overflow, worker-failure blast radius, seam double-free, and forgotten lock guard.

A previous study already estimated that boundary from a list, not from programs that were built. It took the Common Weakness Enumeration (CWE), a long list of bug types. It marked each type as one that safe Rust would stop or would not. It then counted real product bugs in 68 open-source C projects. 58.2% of those classified bugs sat in types that safe Rust would have made almost impossible [26]. That share is a best case: it assumes the new code stays in safe Rust. A type counts as stopped if the mapping says so, even if no compiler has rejected a program. RQ2 asks the same question of programs that are built and run. That is how it can say when the types still open are caught, and what it costs to catch them.

# 2 Background: Systems Languages for Latency-Critical Software

## 2.1 RAN control-plane systems and the gNB-CU-CP

The next-generation radio access network (NG-RAN) is the setting for this comparison. Event rates are high: UEs move, and signalling can spike. Latency bounds are tight and must stay stable. The 5G service and RAN specs state those bounds as key performance indicators (KPIs) [5, 27]. The software lives for a long time and keeps growing. Many threads run at once. Those bounds are on the latency of a signalling call. A late call still misses them when the average call is on time. So the worst case and the tail of that latency matter more than the mean. That is why this thesis measures the tail.

The gNB is split on two axes. The first split is central versus distributed. The gNB runs a protocol stack: a set of layered radio protocols from the air interface up to RRC signalling. A central unit (gNB-CU) hosts the higher layers. One or more distributed units (gNB-DU) host the lower layers. Those layers sit close to the radio and have tight low latency requirements. The second split is control versus user plane, inside the central unit. It splits the control-plane function (gNB-CU-CP) from the user-plane function (gNB-CU-UP). These parts talk over standard links. NG goes toward the 5G core, F1 sits between the CU and the DU, and E1 sits between the CU-CP and the CU-UP [1, 2, 28]. Open-RAN pushes the same links further. Each part can then come from a different vendor and run as software on ordinary machines [3]. Figure 2.1 shows the part studied in this thesis, UE Context Management (UECM), inside the gNB-CU-CP. UECM owns the per-UE control-plane state. It also handles RRC signalling and mobility. These are integral control-plane functions, which makes UECM a representative stateful subsystem. It does not stand for the control plane as a whole.

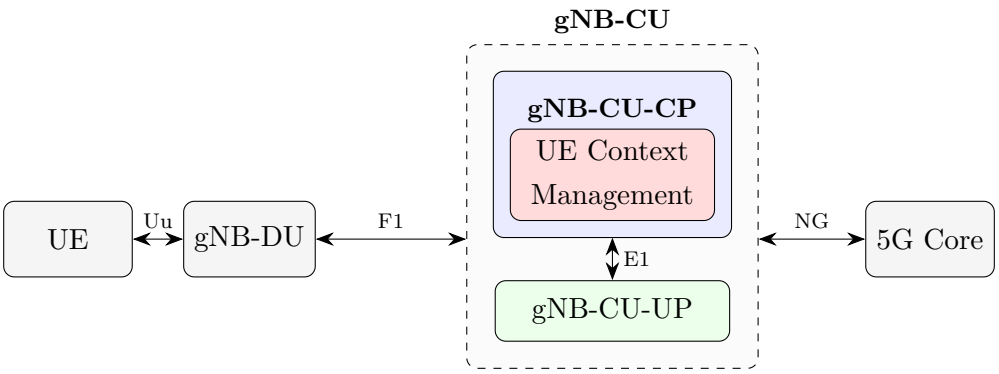



**Figure 2.1:** Where the studied subsystem sits in the 5G NG-RAN. UE Context Management is highlighted inside the gNB-CU-CP.

## 2.2 Research gap

The RAN control plane has many threads and tight latency bounds. Tail latency under too much load, not mean throughput, sets the quality the network delivers. C++ gives the control and speed the domain needs. But it leaves whole classes of memory and thread bugs to detection at runtime. Rust promises to block those classes of errors at compile time at no run-time overhead. It also offers a step-by-step adoption path through the C application binary interface (ABI).

Five vendor-hosted Master's theses, all done at Ericsson, have compared candidate languages for RAN and telecom software. One weighs run-time cost against how fast people can build, across C++, Rust, and Go [29]. One asks whether Rust could replace C in RAN node management [30]. One builds a packet processor, one thread per core, in async Rust. It measures that processor against the Data Plane Development Kit (DPDK) program it mirrors [31]. One measures median and p99 latency for control-plane steps across five async frameworks [32]. The last builds matched C++ and Rust versions of one program. That program turns a packed Abstract Syntax Notation One (ASN.1) message back into a readable form. It compares them with statistical tests [33]. All five are directly relevant. They do not all favour Rust. Olsson and Stenström's packet processor reaches a fifth to a half of DPDK's throughput, with far worse latency [31]. Arvidsson and Kaller's Rust p99 rises more steeply past a threshold rate [32]. Karlbäck and Orö pick C++ where speed is critical. They still find Rust lightest on peak memory [29]. Olsson's management software is as fast as C or faster. It uses more peak memory [30]. Emberg and Wadsten's pair is comparable in one session. It is 85–91% worse in the next [33].

Three of them change the language together with the run-time and the thread model. A

measured gap then cannot be blamed on the language rather than on the framework [29, 31, 32]. In that packet-processor study the DPDK program does two to five times as much work [31]. A compiler does not move throughput that far. The run-time does. Olsson's management study and Emberg and Wadsten do not change the run-time with the language [30, 33]. But they report run time and peak memory, not tail latency under rising load. Emberg and Wadsten come closest in method to this thesis. Their program keeps no state between calls. The service in this thesis stores state and uses many threads. Figure 2.2 places the five vendor theses beside the general-purpose comparisons [34] and the matched systems-software pairs [35, 36]. Each of these groups leaves out at least one thing the figure asks for. No group keeps the design fixed, measures the tail, and drives the service into overload.

The place these studies come from is part of the gap too. All five studies come from one vendor. Their conclusions have never been tested against another firm's software, targets or house rules. So what is known about this question is what one firm's frameworks say about it. The published record on the other side is thinner. There is no study of Rust among the indexed papers of the vendor hosting this work. The nearest peer-reviewed stance from that side argues for making memory-error sanitisation practical. It does not argue for replacing it with a compile-time guarantee [37]. That is the strategy this thesis tests against.

What is missing from prior work is a direct, controlled test of a typical control-plane part. The open questions are these. Does Rust hold its tail latency under too much load against modern C++ when the thread model, operations, and data layout stay fixed? Which bug classes does it remove, and at what stage are the rest caught? What does its safety model cost in throughput and in CPU and memory? What does step-by-step adoption cost in practice? Closing this gap is the purpose of the four research questions. The next chapter grounds them in the part under study, the bug classes, and the timing at stake. It also sets out the method.

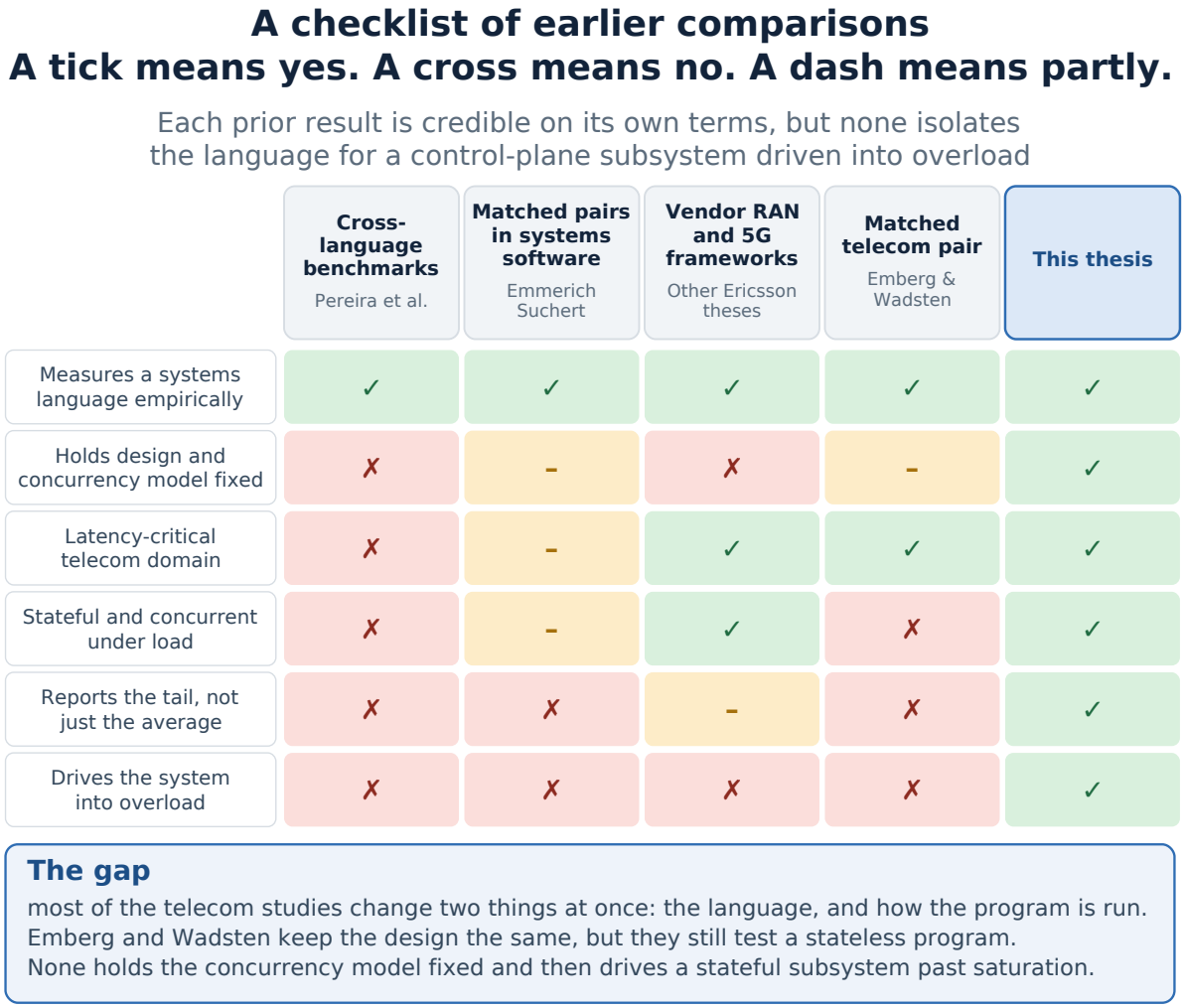



**Figure 2.2:** A checklist of earlier comparisons. Each row is a property a controlled answer would need, and each column is a group of studies. The vendor-RAN column is the other Ericsson theses. Emberg and Wadsten have their own column because they keep the design the same in both languages.

# 3 Research Methodology

The last chapter set out the gap. This chapter says how the study closes it.

## 3.1 The UE Context Management subsystem

The part under study is a UE Context Management service. Its operations follow the UE-context lifecycle, RRC procedure execution [4], and mobility and handover that the 3GPP design assigns to the control-plane part of a split gNB (Section 2.1). Figure 3.1 shows the UE-context lifecycle both versions encode the same way. The mixed workload of Section 5.2 walks each UE through that machine.

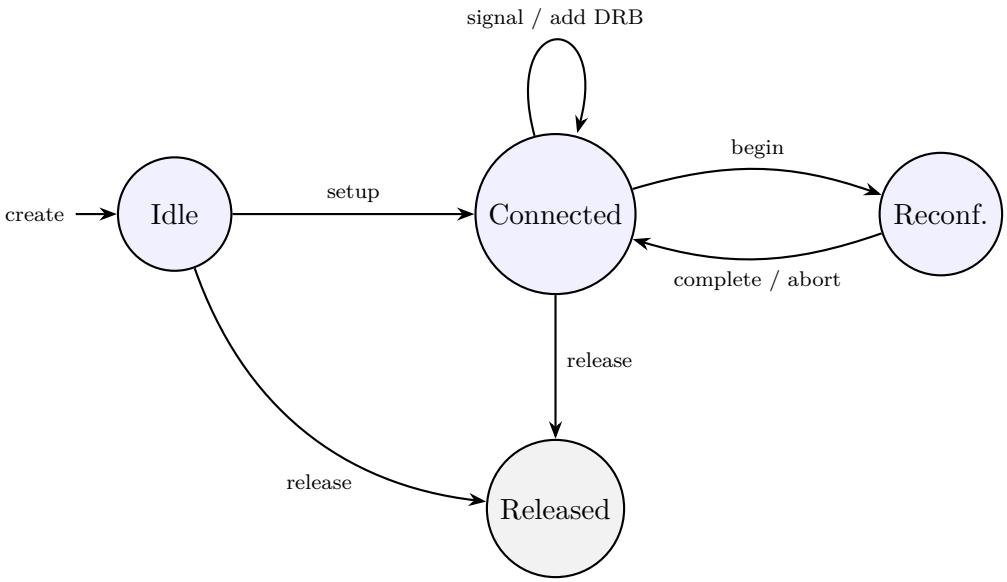



**Figure 3.1:** UE-context lifecycle state machine. Operations invalid for the current state return an error instead of changing state.

UECM is core RAN layer-3 work. It has the traits that make live control-plane code hard. Each UE has state that changes, the operation lifecycle is strict, and many UEs run at once. Delay targets are the ones the 5G system sets for control-plane procedures [5, 27]. Findings about how a language handles those traits are most relevant to other stateful, multi-thread control-plane services of this kind. The part is also small enough to specify in full and build twice. That is what makes a parity-tested comparison possible.

The interface is small and built around the lifecycle. It can create and release a context, set up a connection, handle an in-session signalling message, add a data radio bearer,

begin, complete, or abort a reconfiguration, and look up a context's state under load. Each operation is defined for each state of Figure 3.1. An illegal operation returns a typed error and leaves the context unchanged. That closed set lets the same tests and the same workload drive both versions along the same paths.

RQ2 needs a catalogue, not a loose list of hazards. One is built from this operation set: 14 classes in two groups of seven. Seven come from inside the region safe Rust claims to govern. Seven were chosen because safe Rust is expected not to prevent them. Each has its expectation recorded before the programs were run. Section 5.3.4 reports what each one did.

The tail is the metric because of two traits of this part. One user-visible action, a call setup or a handover, becomes many dependent control-plane operations. The slowest member gates completion. A busy site drives thousands of such procedures per second. So a far-tail point is reached often enough to matter. The failure shape is the second reason. Under an open-loop clock, achieved rate can track offered load until either the service or timed producer work becomes limiting. Latency then rises sharply. That is why this study sweeps past the apparent saturation point and audits what the timed path contains.

## 3.2 Research approach

The study runs experiments. It also builds the thing it measures: a matched pair of versions, plus the harness that drives them. Both are built from one frozen spec. Evidence comes from three separate methods: a load sweep contributes to RQ1 and RQ3, controlled fault injection answers RQ2, and an FFI migration prototype answers RQ4. A weakness in one of them, such as how much the host can move the tail, does not carry into the rest of the claims.

Each question asks how one outcome depends on one variable, the language. The language has to be the only planned gap between the two objects under test. That rules out the most common way to compare languages. In that form two programs written on their own are measured, and the gap is taken to be the language. Such a gap also carries the design, the libraries, and the skill of their authors. Building both sides from one spec removes that confound. Where a question can be answered with numbers, it uses numbers. Where it cannot, the answer is still by a fixed plan, in words. That is how this study treats the kind of guarantee in RQ2. RQ4 asks about the design cost of the application programming interface (API).

The closest prior work is the five vendor-hosted theses of Section 2.2. They appear in

no citation database. They had to be found by searching each university's own archive by hand. The same searches found no Finnish thesis on Rust in a radio-access setting. They found no indexed paper mentioning Rust from the vendor hosting this work. The search returned no study that measures the tail of a Rust and a C++ version of the same stateful, multi-thread control-plane part under too much load.

Table 3.1 maps each research question to the method, artefact, data and metric that answer it. RQ1 and RQ3 share H1, worded to cover a speed tie in both the latency and the resource sense. That wording turns out to be the hypothesis's main weakness. RQ2 does not depend on the host. RQ4 depends on it along one axis. Build time is a timing. So it is read as a ratio between two toolchains measured in turn on one machine.

**Table 3.1:** How each research question is answered. RQ1 and RQ3 carry two methods rather than one. Below saturation the sweep measures the round-trip a deployment would experience. The split of the core from the handoff measures the part the language can own, while a same-host control tests whether high-load recorder work contaminates the sweep.

| RQ | Method | Artefact / data | Primary metric |
|---|---|---|---|
| RQ1 | Load sweep to overload, interleaved repeats | Benchmark harness, latency histograms | median, p99/p99.9/p99.99, and maximum vs. offered load |
| RQ3 | Same sweep, resource capture | `getrusage` samples per run | peak resident set size (RSS), CPU, throughput |
| RQ1, RQ3 | Core called directly, then through the service | Matched microbenchmark pair | operation vs. cross-thread handoff |
| RQ2 | Controlled fault injection | Defect catalogue, three C++ sanitisers, `rustc`, Loom/Shuttle, Miri | compile-time / runtime / undetected |
| RQ4 | FFI migration prototype | C++ host + Rust C-ABI boundary | boundary size, `unsafe` scope, per-call cost |

## 3.3 Methods per research question, in outline

RQ1 and RQ3 are studied with an open-loop load sweep from light load to well past saturation. Open-loop means the generator keeps to its send schedule instead of waiting for replies [38]. The four build variants are interleaved inside each repeat. A second timing splits the core operation from the cross-thread handoff around it. The end-to-end round-trip bundles two costs of two kinds. Only one of them can differ between the languages.

A third timing takes the contended queue and the reply handle on their own. A same-host control then writes the histogram fill after the timed window, holding the pin and `nohz_full`. That control isolates the fill. Clock A, which drops the fill, was not repeated on this host.

RQ2 is answered by controlled fault injection [39]. Each defect class is a small program of its own in both languages. The C++ side is built and run under AddressSanitizer. It is also run under ThreadSanitizer and UndefinedBehaviorSanitizer. The safe-Rust side is compiled and, where it compiles, run under a bounded timeout. No static analyser is run, on purpose. The C++ columns then report the tooling this study ran. They do not report a ceiling on what C++ tooling could find. RQ4 is answered by a working prototype in the direction a real migration takes. A C++ host calls an unchanged Rust core across a C ABI. This boundary is called the FFI seam here. It is the thin layer that converts between the C++ host and the Rust core. The per-call cost is measured on a real operation and not inferred from an empty crossing. Memory bugs that cross a language sit at that seam [40].

## 3.4 Fairness as a controlled variable

A language comparison is only as good as the variables it holds fixed. The spec is frozen before either version exists. The thread model, the allocator and its arena count, the opt level, the hash function, the histogram method and the load schedule are matched across the two sides. They are never tuned per side. Where the two languages could not be written alike, the gap is logged instead of resolved. The two sides are shown to match by diffing match logs operation by operation. For a given seed they also execute the same operation order. The four variants take turns on the same pinned cores inside each repeat. That keeps recent cache contents and recent branch guesses close, because the same work runs one after another. The test does not empty the cache between variants. It also does not first train the CPU on those branches. It holds the work and the cores fixed. It does not hold the CPU's recent memory and branch history fixed.

## 3.5 Statistical treatment

The statistical design is fixed in advance. The family of tested comparisons is listed ahead of the campaign. So a load picked after seeing a sweep cannot enter it. Each comparison is an exact paired permutation test over 20 interleaved repeats, with a bootstrap confidence interval [41], Holm-corrected across the family [42]. The floors the test can reach

(0.0000019 uncorrected, 0.000027 corrected) are quoted so that a null can be told apart from a math limit. Representative-load throughput is reported but not tested because it is pinned to the offered rate. Jitter is outside the research questions. The maximum is too sensitive to one disturbed operation for pairing to rescue it. The pooled p99.99 far tail has the operation as its unit, so it has no across-repeat interval to test. No latency percentile is tested after saturation because cumulative schedule drift makes it a lower bound rather than a service-time estimate.

## 3.6 Reproducibility and research artefacts

A third party can rebuild the results without access to Nokia's private code. The versions are driven entirely by the public frozen spec. The benchmarking harness, the fault catalogue and its runner, and the FFI prototype are published with the thesis. Each plot of measured data is rebuilt by one script from the raw logs. So is each quoted number in the results. The script is `bench/refresh_thesis.sh`. That claim does not cover the hand-drawn schematics, which explain a concept rather than report a result. And running the same scripts over another host's campaign rebuilds the artefacts by the same procedure, not with the same values.

The logs that script reads are part of the package. Each campaign has its own script and its own macro prefix, so a figure from one cannot reach a sentence about another. Each run's host setup is recorded and stored beside the campaign data. Nothing in the results chapters is typed from those files by hand.

# 4 Design and Implementation

Nothing is built until the spec is frozen. It fixes the public API, the UE state machine, the data layout and the steps on it, the locks and shard size, and the thread model. Both versions follow it. The test then times the language, not a gap of design or of skill. The spec is itself an output of the study. It is the main control against writer bias.

One thread model is used on both sides: shared-nothing and thread-per-shard. That layout is known to change tail latency by itself [43]. The layout is held fixed as a fairness control. A production gNB-CU-CP schedules UE work differently. The id space is split into shards. Each shard is owned by one worker thread. That thread alone reads or changes it. So no lock guards context state. Callers send a request to the owning shard's bounded queue. They wait on a one-shot reply handle. The queue blocks when it is full. That is on purpose. A drop rule would hide overload. This study needs overload to show up as latency. The only shared mutable state either version allows is one atomic counter. No control flow depends on its value. Figure 4.1 shows the data flow. There is one multi-producer, single-consumer (MPSC) queue per shard. Its worker drains it first in, first out (FIFO).

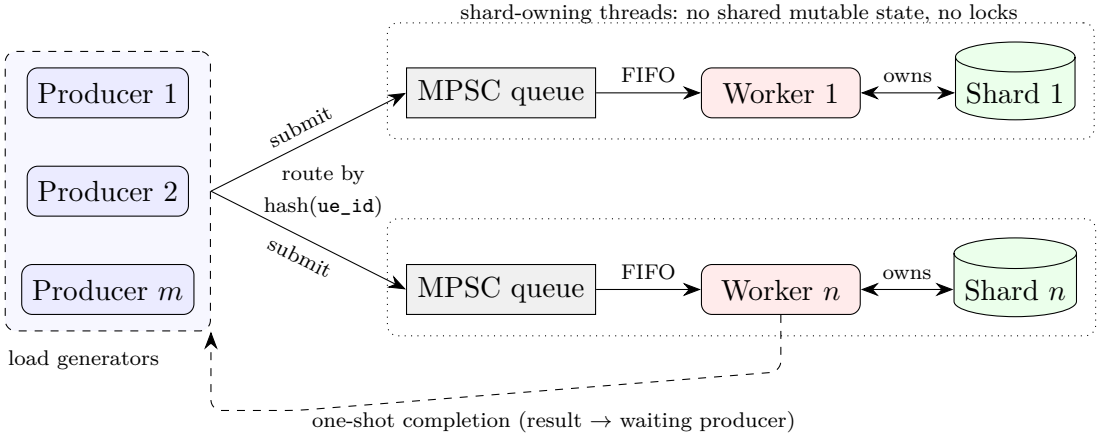



**Figure 4.1:** Shared-nothing, thread-per-shard architecture. Producers submit to a bounded MPSC queue per shard, and that shard's worker drains it.

Fairness is kept below the design as well as in it. Both builds link the same `jemalloc`. Both keep a system-allocator build as a control. Both compile at the same opt level. Both record latency through one high-dynamic-range (HDR) histogram set up the same way [44]. Hashing is pinned on both sides. SplitMix64 assigns the shard [45]. FNV-1a, the Fowler–Noll–Vo hash [46], is used inside the map. C++'s default hash of an integer is often just the number itself. Rust's is strong on purpose. Left as they came, the two

standard libraries would have been the measurement.

Where the languages cannot meet a need the same way, the gap is written into a forced-idiom log and then measured. Two of those gaps were candidates for the campaign-clock overload result of Section 5.3.2. One is the bounded queue. Rust takes it from `sync_channel`. C++23 still has no such primitive, so C++ builds one from a mutex and two condition variables. The other is the reply handle, `std::promise` against a fresh `mpsc` channel per request. Tests of those primitives and a replacement reply handle were run on an earlier host. They do not explain the smaller lead that remains on this one once the fill moves. A third carries the core-operation result of Section 5.3.1: the per-shard container is each language's own standard hash map. A fourth is the request encoding. C++ has no payload-carrying enumeration in this idiom, so a request becomes a tagged struct holding each field any operation might need, 160 bytes against Rust's 96. The net result is a Rust core 20% longer in code lines. Its branching complexity is 13% lower [47]. The Rust library contains no `unsafe` at all.

Parity is tested and not assumed. Language-neutral vectors run in both versions. Their logs are diffed operation by operation. A cross-diff cannot tell "both correct" from "both wrong in the same way". A third version in Python acts as an independent oracle. It has to reproduce all 223 hand-derived expectations before it may generate any of its own. A campaign of 9 000 generated operations found no mismatch. The C++ side runs under AddressSanitizer, ThreadSanitizer and UndefinedBehaviorSanitizer. The Rust side runs under Miri. A clean sanitiser result is bounded by the code that actually ran.

# 5 Experimental Setup and Evaluation

This chapter starts with the test hosts and the work they run. Results then follow by load, then by the rest of the questions.

## 5.1 Testbed setup

Some numbers in this chapter come from the tools and the code. Other numbers depend on the host they ran on. The reported campaign comes from one bare-metal host, a single-socket AMD EPYC 7443P. Each core runs one thread. Extra threads on a core are off. The host runs Ubuntu 24.04.4 LTS on kernel 6.17.0-23-generic. Linux can raise and lower each core's clock as load and heat change. The clock is held fixed. The governor is performance, and boost is off. The test may use only cores 2–17. There is no hypervisor. The program runs on the host itself, not inside a virtual machine. On those cores the kernel turns off its timer tick (`nohz_full`). If left on, those ticks would show up in the latency numbers. The study does not use `isolcpus`. That flag also stops the kernel sharing work across the allowed cores. Each test thread then lands on one core.

The build tools are not the ones on the host. Both sides are built and run inside an image that carries Rocky Linux 8.10 (Green Obsidian) with `g++ 15.2.1` and `rustc 1.97.1`. The image pins the `gcc-toolset` major and not the point release. The earlier sweep on a virtual machine then built with `g++ 15.1.1`, one GNU Compiler Collection (GCC) 15 release earlier. An image still uses the kernel of the host it sits on. The two sweeps then differ in the hardware, in the kernel (6.17.0-23-generic against 7.0.2-1.el8.elrepo.x86_64), and in that compiler release.

Until late in the work the sweep ran on a Kernel-based Virtual Machine (KVM) on an Intel(R) Xeon(R) CPU E5-2680 v4, with none of the core controls above. The capture records the hypervisor as kvm, the guest userland as Rocky Linux 8.10 (Green Obsidian), and the guest kernel as 7.0.2-1.el8.elrepo.x86_64. On that host the 95% band on one variant's p99 spans 1480 $\mu$s. The four variants' p99 values differ by 293 $\mu$s. The noise on one variant was larger than the gap between all four. That sweep is kept. It uses the same design as this one. The two can then be set next to each other to ask which findings

belong to the code. A third sweep on a second bare-metal AMD EPYC 7443P uses a new shard count, load grid, run count and tools. It tests whether the findings hold under those changes as a set, not any one of them on its own. Appendix B itemises the three hosts and the campaign parameters field by field.

## 5.2 Workload and measurements

The work is made for the test. The steps it takes come from the 3GPP rules [1, 4]. A driver that always sends the same steps walks each UE through the connection lifecycle those rules define. The *signal* work first connects a set of UEs, then sends the same signal call again and again. That picks out the common path. The *mixed* work walks each UE through context creation, setup, signal, bearer setup, reconfiguration or handover, and release. A fixed random order makes both sides see the same counts of each call kind for a given seed. That is also a check that the two sides did the same work. Figure 5.1 shows the two loads and the steps the mixed one walks.

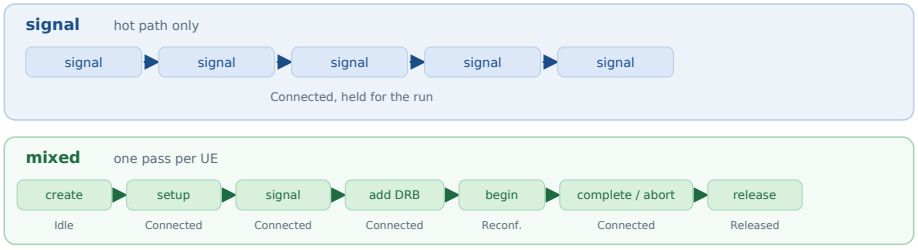



**Figure 5.1:** The two workloads, and the lifecycle the mixed one walks. Signal repeats the common path, and mixed walks each UE through the procedure sequence drawn below.

The steps match the 3GPP rules. Those rules do not set the mix. A real cell's handover rate rests on the site and the time of day, and no public figure fixes it. A mix that does not match a real cell moves both sides by the same amount. It limits how far the numbers travel. It does not make the test unfair. The sender keeps to its plan even if the service is late (open-loop). The histogram is corrected so a stall does not hide missing replies (coordinated-omission bias) [38, 44]. Each load is run 20 times. The four language and heap variants are interleaved inside each run. Each point is a median with a bootstrap 95% band [41, 48].

The load used for the resource plots is 56 kilo-operations per second (kops/s). It is a chosen value, not one made from the sweep. The first rule this study used was the peak load whose p99 stayed under 5 ms. On the virtual machine that rule landed on the noise of the host. On a quiet host it climbs until the service can no longer keep up. It is kept

only as a check. The lowest finished rate among the four variants is 392 kops/s, reached when the sent load is 392 kops/s.

## 5.3 Results

The safe code results and most of the migration results do not depend on which host was used. The speed and RSS results do. Each figure below was timed on the host of Section 5.1 unless the caption says so.

### 5.3.1 Below saturation (RQ1, RQ3)

At the representative load of 56 kops/s, where every variant keeps up with the sent load, the four variants' medians lie within 1.6 $\mu$s of each other, 2.2%. A paired test still finds a gap of 1.5 $\mu$s in Rust's favour under both heaps. Figure 5.2 shows the sweep behind that. Higher tails can now be read as well. The widest band on one variant's own p99 spans 0.8 $\mu$s. The four variants' p99 values differ by 1.8 $\mu$s. So the noise on one is smaller than the gap between them. On the virtual machine the same two sizes were 1480 $\mu$s against 293 $\mu$s, the other way around. Both p99 rows and both p99.9 rows clear the Holm correction, 4 rows in all, in the same way as the median. Table 5.1 reports p99.9 and the maximum at the chosen load. The maximum is kept out of the tested family, as the pooled p99.99 is.

**Table 5.1:** Per-operation latency, in microseconds, at 56 kops/s. The median over 20 repeats sits above its bootstrap 95% interval. †The p99.99 column is pooled across repeats, so it has no interval. The maximum has a repeat-wise interval but stays outside the tested family because one disturbed operation can determine it.

| | | | (µs) | | |
|---|---|---|---|---|---|
| Variant | p50 | p99 | p99.9 | p99.99† | max |
| C++ (system alloc) | 73.9 | 76.9 | 81.0 | 110.0 | 226.4 |
| | [73.9, 74.0] | [76.4, 77.2] | [80.7, 82.0] | | [221.1, 239.3] |
| C++ (jemalloc) | 73.8 | 77.1 | 80.2 | 143.0 | 227.8 |
| | [73.7, 73.9] | [77.0, 77.1] | [79.9, 80.9] | | [217.2, 244.4] |
| Rust (system alloc) | 72.7 | 75.3 | 77.1 | 87.9 | 100.3 |
| | [72.6, 72.7] | [74.9, 75.6] | [76.9, 77.9] | | [97.4, 160.8] |
| Rust (jemalloc) | 72.3 | 75.7 | 77.8 | 81.3 | 104.4 |
| | [72.2, 72.3] | [75.6, 75.8] | [77.6, 77.9] | | [94.1, 141.8] |

The far tail can be read here too. The runs' histograms are pooled, which gives 3 360 000 to 3 360 063 calls per variant. The two Rust variants sit below both C++ variants. The

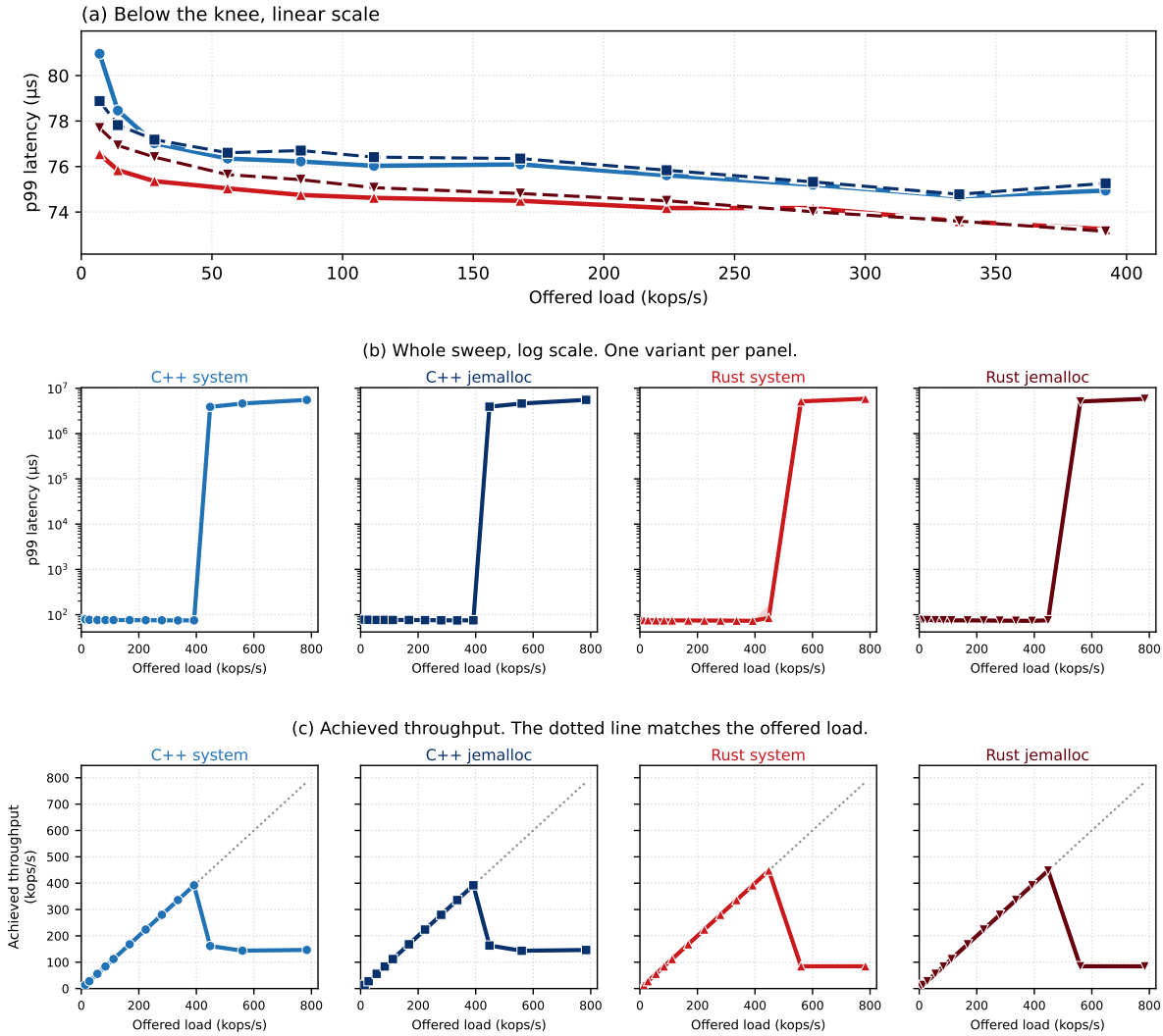



**Figure 5.2:** Panel (a) is p99 below the knee, on one linear scale, through 392 kops/s. The four variants sit within a few microseconds, so they are drawn together. In (b) and (c) each variant has its own panel. Panel (b) is the tail on a log scale across the whole sweep. Panel (c) is achieved throughput, and the dotted line matches the offered load. Lines are medians over 20 repeats, and the bands in (b) and (c) are bootstrap 95% confidence intervals.

widest pair differs by 62 $\mu$s. The slowest Rust variant still sits 22 $\mu$s below the fastest C++ one. A second bare-metal host at another shard count agrees, running 147–228 $\mu$s for the C++ variants against 88–115 $\mu$s for the Rust ones, and the ranges do not meet. A second limit does not come from the host. No host would remove it. An end-to-end round-trip spends only 0.250% of its time on the work the language does. The rest is passing the request to another thread, and that time is charged to the operating system. When the work inside the language is this small, time spent crossing from thread to thread dominates [49]. Figure 5.3 draws that split. This sweep cannot bound a language effect either way, which Figure 5.4 sets beside what the sweep can still show. Timed on its own,

the core call is 1.6× faster in Rust. The test does not trace the cause. The stock hash map is one candidate. Compiler code generation may also own part of the gap.

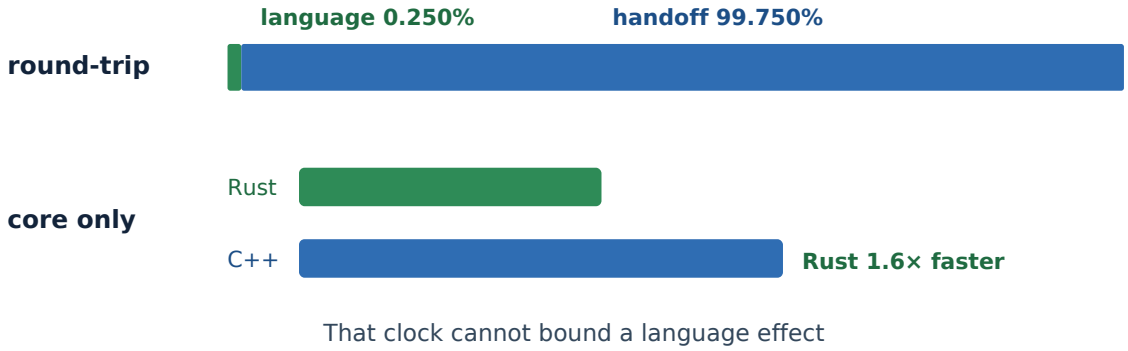



**Figure 5.3:** Where an end-to-end round-trip's clock goes. The language share is 0.250%. A clock that spends the rest on handoff cannot bound a language effect.

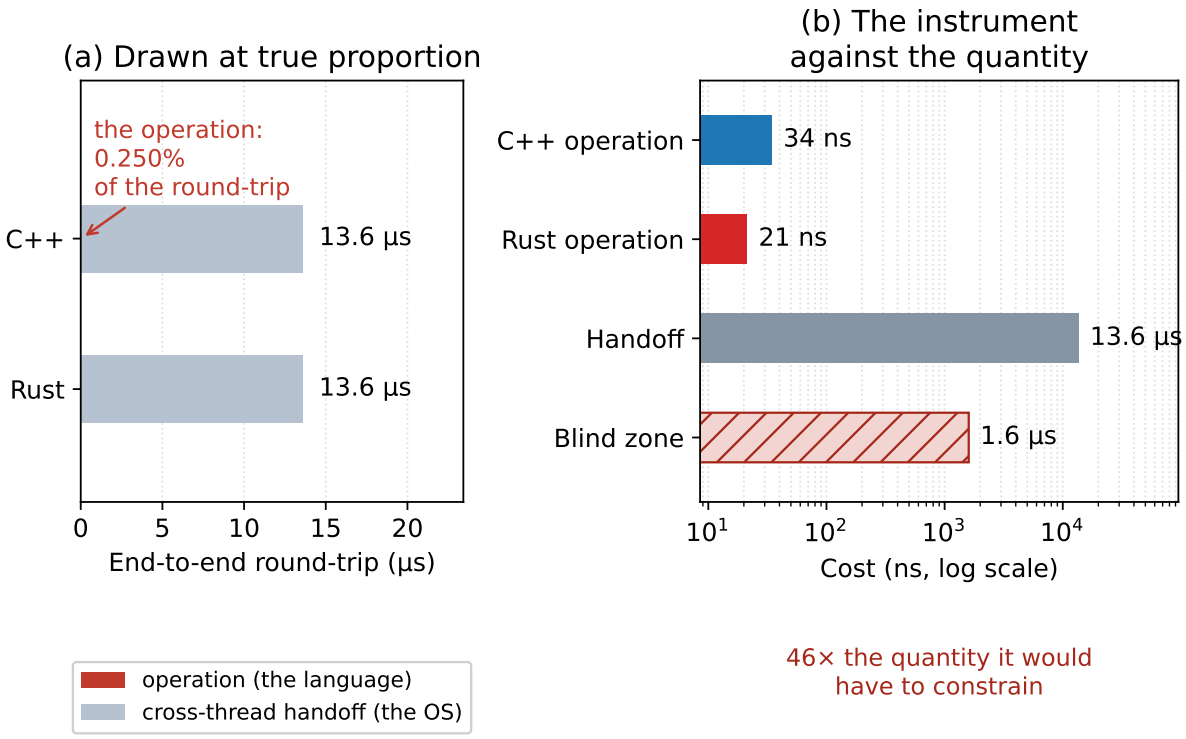



**Figure 5.4:** Where an end-to-end round-trip's cost goes. **(a)** The round-trip at true proportion. **(b)** Both operation measurements sit inside the sweep's blind zone by a factor of 46.

### 5.3.2 Past saturation (RQ1)

Under the campaign clock both languages keep up through 392 kops/s and 392 kops/s, the same load. At 448 kops/s the split is inside Rust. The jemalloc heap still serves 448 kops/s.

The system allocator median is 214, and 9 of 20 repeats on that heap still meet the offered rate. That rung was found after the data and is shown, not tested. Figure 5.5 maps the representative load, the shared tracking load, the allocator split, and the overload point.

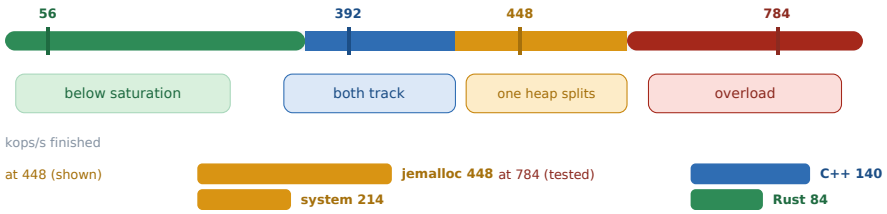



**Figure 5.5:** Four load points, from a light load to overload. Operating points on the offered-load axis, spaced for reading, not to scale. Ticks are 56, the shared tracking load 392, the allocator split at 448, and 784 kops/s. Bars at the split are the two Rust heaps on the campaign clock. Bars at 784 are the two languages, also on that clock.

Deeper, at 784 kops/s sent, C++ still finishes 140 kops/s where Rust finishes 84, a factor of 1.7, in 20/20 runs under both heaps. The Holm-corrected $p$ is as small as this test family can go. The slower variant uses 1.7 times the CPU for each call it finishes. That quantity belongs to the whole campaign process, including the recorder, and does not identify service work.

Clock B repeats the same grid on this host, over 1120 runs. The processor is AMD EPYC 7443P, the pin is 2–17, and `nohz_full` is 2-17. Those match the campaign. Only the fill moves, to after the timed window. When a sender falls behind, the harness still writes up to 10 000 extra samples for each finished call, but it writes them outside the window. Clock A was not repeated here. It remains an earlier host. Its percentiles stay coordinated-omission biased, so they are not an RQ1 tail result.

Moving the fill removes both campaign patterns. At 448 kops/s, C++ serves 416 kops/s and Rust serves 448. Rust still tracks to 448 kops/s and C++ to 392. At the top load Rust is ahead on both heaps. On the system allocator C++ finishes 0.95 of Rust's throughput, 412 against 434 kops/s, at $p < 0.001$. Under `jemalloc` the figure is 0.92, 413 against 450, at $p < 0.001$. These tests sit outside the pre-committed Holm family.

What remains is that smaller Rust lead. CPU per completed operation on the deferred plateau is 1.03, with C++ the higher one, against 1.7 while the fill ran inside the window. The large ratio is the recorder. The smaller lead is not that ratio. This visit took no allocation profile, so the lead is reported and not explained. Queue and channel substitutions from an earlier host do not explain it either. Percentiles on Clock B are schedule drift, not service time. The campaign stays the pre-committed compared pair. Figure 5.6 compares

both clocks on this host and keeps the two heaps apart.

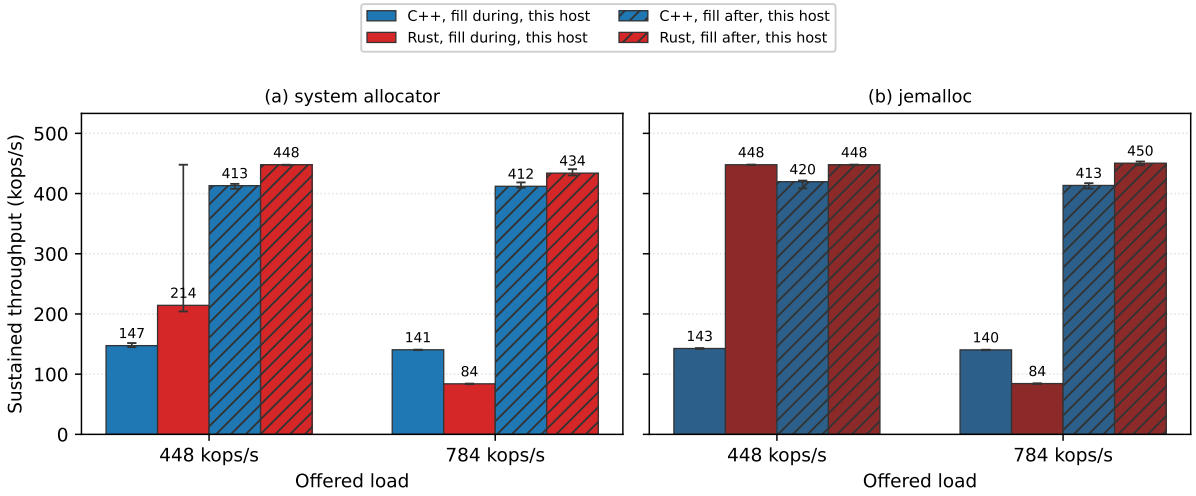



**Figure 5.6:** Throughput at the allocator-split load and at the deepest overload point. Bars are medians over 20 interleaved repeats with bootstrap 95% intervals. Solid bars fill the histogram during the timed window. Hatched bars put that fill after the window, on AMD EPYC 7443P, pin 2–17, and `nohz_full` 2-17. The large gaps move with the fill.

### 5.3.3 Resource use (RQ3)

The four variants differ in RAM and CPU usage even where their median latencies are close. Peak resident set size (RSS) contains two effects of similar size on the reported host. The Rust build uses 1.6–2.5 MiB more peak RAM than the C++ build under either heap. Linking `jemalloc` costs 1.5–2.3 MiB in either language. Figure 5.7 shows both effects. The language effect keeps its sign and MiB scale on all three hosts, but not one exact range. It spans 1.8–3.5 MiB on the guest and 0.7–1.8 MiB on the second bare-metal host. The allocator effect's size does not travel either: on the virtual machine it was 13 times the language effect, 32.5–34.3 MiB. Their ratio is therefore a host property. Allocator accounting on the second bare-metal host locates the language gap outside the heap. Rust has +2.07 MiB more arena metadata but −0.16 MiB less live allocation and −1.92 MiB less allocator-resident memory, while the process peak remains +1.74 MiB higher. The extra RAM is a fixed process cost, not more live application data. CPU is a separate result. At 56 kops/s Rust uses at most 6.2 percentage points less. At 784 kops/s the slower campaign-clock variant instead uses 1.7 times the CPU per finished call. On this host the deferred-fill sweep does not reproduce that large ratio. CPU per completed operation there is 1.03, with C++ the higher one. The large ratio is the recorder. The smaller throughput lead remains, and this visit does not explain it. Counters on the second bare-metal host cover only the isolated core call. Rust retires 11% fewer instructions and

finishes in 15% fewer cycles, while missing cache 29% more often per thousand instructions. Those counters explain the isolated core result, not the overload floor.

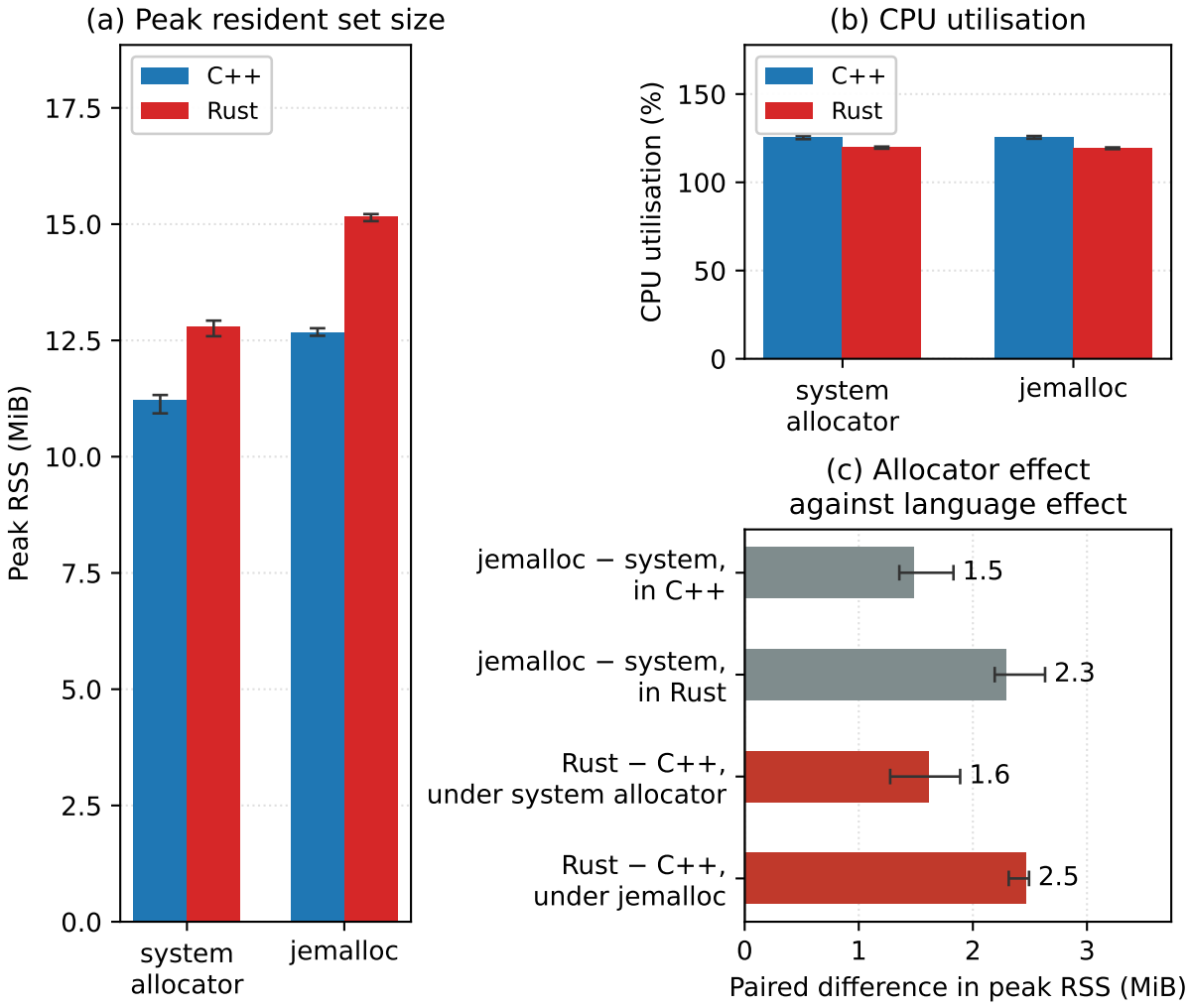



**Figure 5.7:** Resource use at 56 kops/s, grouped by allocator. Bars are medians over 20 repeats with bootstrap 95% intervals, and CPU in (b) is summed across threads.

### 5.3.4 Safety via fault injection (RQ2)

Safe code is checked by putting 14 defects in. Seven come from inside the zone Rust claims to govern. Seven were chosen because safe Rust is meant *not* to stop them. Each one of the first seven is a compile error in Rust. In C++ each is valid code. The defect shows up only if a test drives it, and no single sanitiser hits them all. The use-after-move is the case where no sanitiser flags anything. A moved-from object is still a valid object. Its value is unset. So there is nothing wrong to detect, and the name is just lost. All seven boundary cases compile in safe Rust and fail. On two of them the C++ tools are the ones

that flag. LeakSanitizer flags the ref cycle that Rust leaks in silence, because leaks are safe in Rust by design [50, 51]. UndefinedBehaviorSanitizer flags the overflow that Rust defines to wrap. ThreadSanitizer catches the lock-order inversion in 3 of 12 runs, a rate that itself jumps. It logs lock orders only from locks that work. If the first inversion waits and never returns, the cycle is never closed, so the tool never flags it.

The seam double-free is the in-guarantee double-free recast through `unsafe`, not a new class. Its C++ half is the same, so only the Rust column moves. Two Rust model checkers do flag both original boundary defects. Loom explores thread orders in a sound way. Shuttle picks the ones Loom cannot reach [52]. Each had to be written using the checker's own parts first, and shrunk to model size. Miri flags the seam double-free, but it reads Rust and cannot run the C++ host the defect sits behind. LeakSanitizer and UndefinedBehaviorSanitizer flag without a rewrite. The two model checkers flag the concurrency boundary cases only after the defect is recast into their parts. Figure 5.8 draws the whole list as an outcome matrix. Resource Acquisition Is Initialization (RAII) ties cleanup to object lifetime. It removes the plain missed-unlock, because leaving a scope destroys the guard and releases its lock. It does not remove the class: both forgotten-guard programs hang, and no side's tools said so.

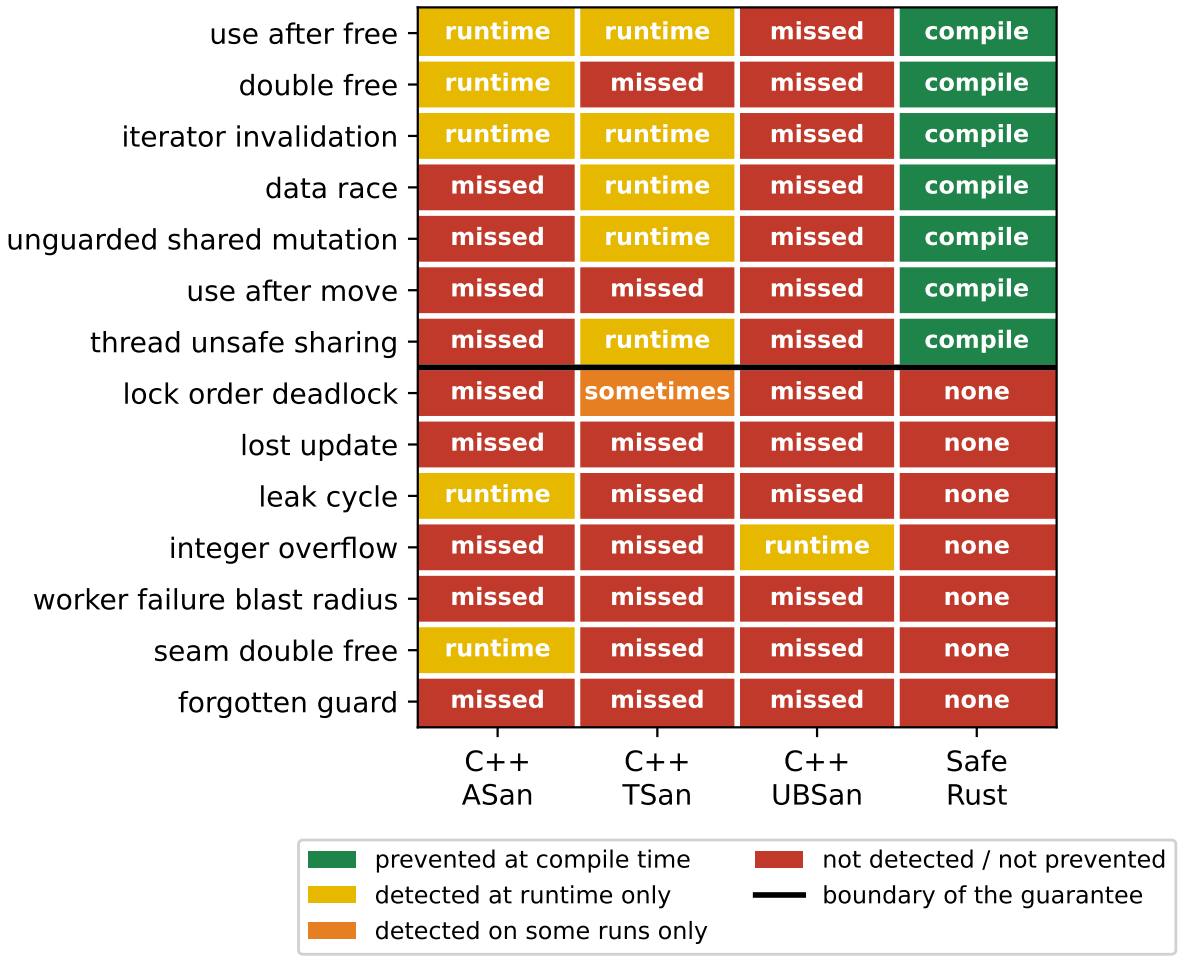



**Figure 5.8:** The fault-injection outcomes as a matrix, generated from the taxonomy log.

### 5.3.5 Migration (RQ4)

The FFI seam, the C-ABI layer between the C++ host and Rust core, is small on each measured axis: 11 C entry points in 231 lines of Rust. All raw-pointer work is kept in 9 functions. The core is left free of `unsafe`. No extra cost is seen against the same signal call made directly. That bounds any per-call tax at 10.91% of that call, and Figure 5.9 shows the structure that bound belongs to. Three adoption costs beyond the seam were measured as well, and two of them go against the usual worry about taking Rust. 54% of the Rust clean build is third-party code. The project's own source builds in 0.8 times the CPU `g++` needs. The 6.8× size gap is a linkage policy that flips when the policy is held fixed. The compared part has no third-party crates and no `unsafe` sites (0 crates, 0 `unsafe` sites).

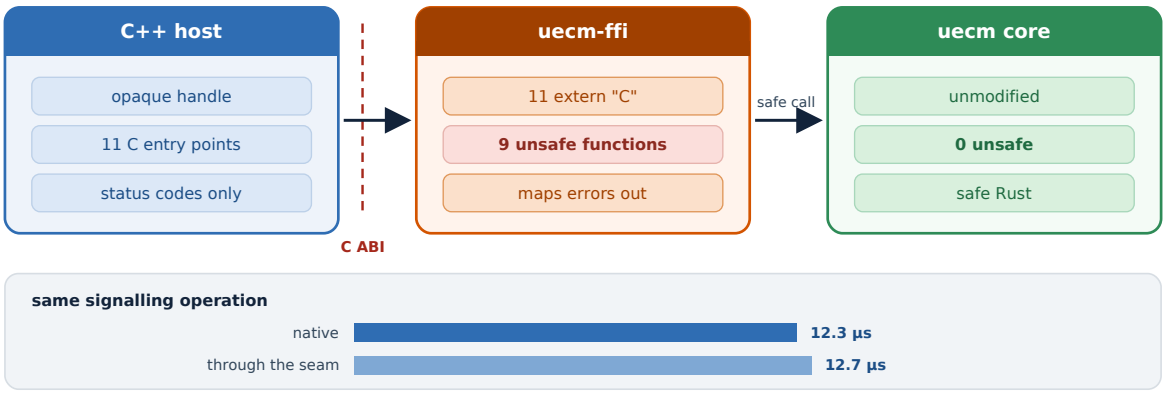



**Figure 5.9:** The C++-to-Rust FFI migration seam. The same signalling operation costs 12.3 µs directly and 12.7 µs through the seam, a paired difference bounded at 10.91% of that call. A do-nothing crossing costs 2.54 ns and bounds only itself.

# 6 Discussion

## 6.1 Summary of findings

At 56 kops/s the variants agree to 1.6 $\mu$s at the median, and the pooled far tail lands on Rust's side. The end-to-end round-trip is only 0.250% language work, so it cannot bound a language effect. The same core timed apart is 1.6× faster in Rust. At higher loads the campaign clock shows an allocator split at 448 kops/s and a 1.7× lead for C++ at 784. Moving the fill after the window on this same host removes both large patterns and leaves a smaller Rust lead. CPU per completed call falls from 1.7× to 1.03×. Neither overload clock supplies service-latency percentiles, so RQ1 has no language verdict after saturation. The RQ2 outcome is complete for the fourteen-class catalogue set in advance and does not depend on the host. All seven in-guarantee classes are compile errors in safe Rust. All seven boundary classes compile and fail in Rust and in C++. RQ3 finds a higher Rust peak RSS under both heaps: 1.6–2.5 MiB on the campaign host, 1.8–3.5 on the guest, and 0.7–1.8 on the second bare-metal host. Below saturation Rust draws slightly less CPU. Corrected high-load throughput gives no general language rank. RQ4 finds a small seam whose per-call tax is bounded at 10.91% of that call. API design and learning remain literature-backed adoption costs that this study does not measure directly.

## 6.2 Discussion on the research questions

RQ1 has two answers separated by regime. At 56 kops/s the campaign resolves median, p99 and p99.9 differences, and the pooled p99.99 lies on Rust's side. The end-to-end clock still cannot bound a language effect because the core call is only 0.250% of what it times. After saturation neither available percentile clock estimates service latency. The campaign includes producer-side fill, while Clock B measures cumulative schedule drift. No overload tail ranking follows.

The campaign throughput shows an allocator split at one rung and a C++ lead later. Clock B, on this same host, removes both large patterns when the fill is deferred and leaves a smaller Rust lead. That isolates the fill as the source of the large gaps. The smaller lead is reported and not explained. This visit took no allocation profile, and the queue and channel substitutions belong to an earlier host. Clock A's lower Rust

p99 is deliberately coordinated-omission biased and is not an RQ1 tail result. Clock B's throughput is a completed-work comparison. Its percentiles are schedule drift. The closest prior work reports similar degradation past a threshold rate and attributes it to stackless `async/await` [32]. That cause cannot carry here because neither side uses an asynchronous runtime. Changing Rust's reply handle, on an earlier host, does not explain the smaller lead that remains here. The campaign ratio therefore remains a result for its harness and does not rank top service rates. The prediction logged in advance had instead named bounds and overflow checking [53, 54]. That part was wrong. At nominal load the median result still fits evidence that Rust's safe checks are mostly absorbed by modern out-of-order CPUs [12, 35].

On safe code some defects are compile errors, so they never become a program that can run. The seven classes Rust does not stop are part of the answer, not a warning added then. A compiler that will not build a program rules the defect out of each run. A sanitiser rules it out only of the runs a sanitiser test happens to take. What is fixed there is the run coverage, not the breadth of the list. Figure 6.1 names the way on each side of that line. Rust reasons about aliasing and not about lock order. A `Mutex` guard whose life the borrow checker tracks well says nothing about whether two threads take their locks in a matching order. Static deadlock checks for Rust are an open research problem [55]. The lost update misses ThreadSanitizer for the same reason it misses the borrow checker. Where each access is synced right, there is no unsynced access to find. In a field study of 105 real concurrency bugs, about a third of the cases that were not deadlocks broke the order the writers meant, not their locking [56]. Taking Rust drops one class of concurrency defect and leaves in place the class that fails in silence. A plan to move that reads "memory-safe" as "concurrency-safe" would mis-budget its review effort. In the corpus of Xu et al., every published Rust Common Vulnerabilities and Exposures (CVE) memory-safety entry but one compiler bug involves `unsafe` [57]. That is why keeping the core free of `unsafe` matters. It is also why unsafe spreading through crates is a review concern [58].

One case reaches past the fault programs. A spec gap let a call be sent outside the service running window. It passed each test, each parity vector and all three sanitisers because nothing ever called that path. What turned undefined behaviour into a catchable failure on the Rust side was a plain bounds check, not a proof. The thing worth planning for is "checked by default", not "ruled out at compile time". The root cause belongs to the method. A ban stated without a way behind it leaves each version to fail in its own idiom.

RQ3 splits by resource and the splits do not point the same way. The RAM cost holds under both heaps and on a second host with a new compiler. Peak RSS is a max for the whole run, so the jitter that destroys a p99 does not touch it. Sized against the heap's own

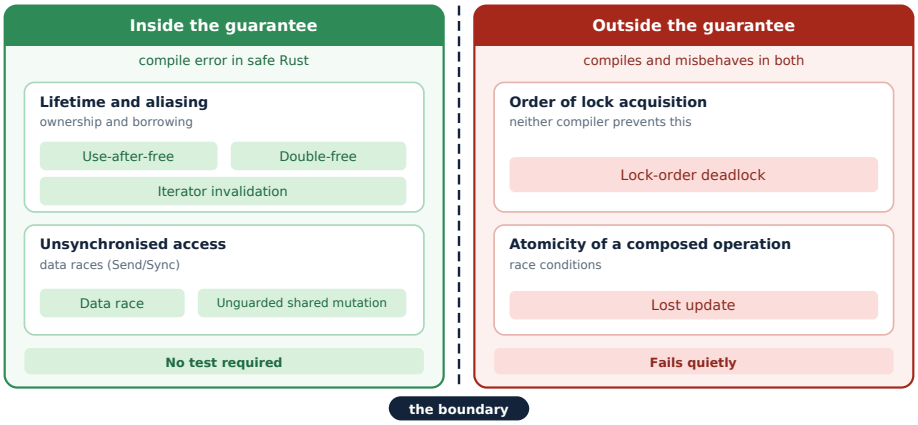



**Figure 6.1:** Where Rust's compile-time guarantee stops, with classes grouped by the mechanism that rejects them.

effect, the two are of the same order on both bare-metal hosts. On the guest the heap's was an order of magnitude larger. The language effect travels. The ratio does not. The quiet host also puts the cost past the heap, by the heap's own counts. One close study reports the other sign, with Rust lowest and the gap growing as concurrency rises [29]. Those two findings agree once the variants are read as two concurrency runtimes, not two sides. Their C++ runs on Boost.Fibers, which gets a stack per fiber. Their Rust runs on Tokio, where a task is a small future on the heap. What they timed is the per-task cost of two concurrency runtimes. Two further studies agree with the way found here [30, 59]. Below saturation Rust draws slightly less CPU. The larger flip under the campaign clock is the recorder. On this host it is absent once the fill moves, and it cannot be assigned to Rust's safe code model. A smaller throughput lead remains.

RQ4 finds a small machine-side seam. A C++ host can call an unchanged Rust core across a thin C-ABI seam whose cost on a real signal call is bounded at 10.91% of that call. Prior work identifies design and learning as human costs [60, 61], but this study does not measure their size. Keeping `unsafe` at the seam bounds what has to be checked, not what can be corrupted. Host and part share one address space [62, 63]. The build and footprint axes point the same way as the seam. A vendor worried about either should audit its crate budget before blaming the compiler. The one axis where C++ leads outright is the edit and compile loop for changes that stay inside one source file. The 6.8× size figure sets two linkage policies, and the order flips when the policy is held fixed.

## 6.3 Revisiting the hypotheses

Section 1.4 set three hypotheses before any run. Saying how they came out is what sets a test apart from a sketch.

**H1** It is not accepted as written. No equivalence margin was defined, so a small significant median difference cannot establish performance equivalence. At 56 kops/s the paired test resolves 1.5 $\mu$s and higher-percentile differences, while the pooled far tail favours Rust. The isolated core favours Rust. After saturation the available clocks do not provide a valid service-latency ranking. The hypothesis combined the round-trip, core code, recorder work and overload behaviour into one quantity that the method cannot support.

**H2** For the seven injected classes of its kind it holds without exception. It fails in the looser reading its wording invites. It is accepted for memory safety and data-race freedom, rejected for concurrency correctness at large, and it is a claim about *safe* Rust only. The seam case shows the same class returning inside `unsafe`.

**H3** Any per-call tax is bounded at 10.91% of that signalling call. This is supported by the upper bound on the paired difference. A non-significant difference alone would not show that the tax is small. The accompanying claim, that the real cost is design and training, is supported structurally and not statistically.

## 6.4 Engineering guidelines for incremental adoption

The rules are the practical form of the findings, and each carries the limit of the finding behind it. Choose a first part for how thin its API is, not for its size. The rewrite of a rich interface is the schedule risk. The edge itself is not. Keep the C ABI in its own crate and the core free of `unsafe`. That keeps unsafe out of the crate chains it spreads through [58]. Leave the concurrency review budget where it is. Prefer a shared-nothing structure in either language. The strongest concurrency result here belongs to the design and to neither compiler. Plan the learning curve as a training cost and track it. Time the part as well as the round-trip. Sweep past the point where the campaign stops keeping up. Audit every operation on the producer's timed path, including histogram correction. On this host, running that work after the timed window removes the large high-load gaps and leaves a smaller throughput lead.

## 6.5 Where this study sits, and what it cannot support

The studies closest to this one are the five vendor-hosted theses of Section 2.2. Their verdicts clash with each other in each way. The one with the other sign on peak RAM fits this study once its variants are read as two concurrency runtimes, not two sides. Four threats still matter, and the first is that one author wrote both versions. The same failure mode flipped a printed verdict in this field: comparable after a code review and then Rust worse by 85–91% after more speed work [33]. The parity suite bounds what the code does and not speed. The second is that the end-to-end clock times the design faithfully and the language hardly at all. Third, the same-host control isolates fill timing. It removes the large campaign gaps and leaves a smaller Rust lead on the deferred plateau. No allocation profile was taken on this visit, so that lead is reported and not explained. Clock A was not repeated here. Fourth, neither high-load clock gives service-latency percentiles. The campaign includes recorder work, Clock A has coordinated-omission bias, and Clock B reports schedule drift. The thesis therefore leaves overload service latency unresolved.

## 6.6 Ethical and professional reflection

The study uses no personal data, no subscriber data, and no operator secrets. All traffic is made for the test. The service is based on live code, not copied from it, so the artefacts can be shared in public. The first duty is not to claim too much. A vendor could read the RQ2 result as licence to describe a moved part as “safe”, and the boundary cases are why that reading would be wrong. A deadlocked control plane is an outage. A lost counter update that stays silent can misinform a mobility decision. Nothing shown here stops either. The second duty is how far the data reaches. Tail latency numbers from any single host are a thinner basis for a production roll-out than they look. That is why the earlier guest sweep is kept, not dropped.

# 7 Conclusion and Future Work

The thesis set modern C++ against Rust. The object is a UE Context Management service. It keeps per-device state, as a gNB-CU-CP does. It looked at four things. Two of them are how late the tail is, and which defects a compile step can stop. The other two are the cost of the safe code model and the cost of adding Rust a piece at a time. Both sides were built against one frozen spec. They were driven along the same code paths. That is what lets a language effect be told apart from a gap of design or of skill.

The safety mechanisms act at different stages. Safe Rust rejects all seven injected memory-safety and data-race classes before they become programs that can run. The same seven compile in C++, where a sanitiser can report a defect only if a test executes the relevant path. All seven boundary classes compile and fail in both languages. On two of them the C++ tools flag what Rust's do not. Rust therefore drops data races and aliasing errors. It does not drop race conditions, lock-order errors, leaks, wrapping arithmetic, or the blast radius of a failed worker. It drops nothing inside an `unsafe` block. LeakSanitizer and UndefinedBehaviorSanitizer flag two boundary defects without a rewrite. The two model checkers flag the concurrency cases only after each defect is recast into their parts.

On migration the measured machine-side seam is small. An unchanged Rust core can be driven from C++ across an FFI seam, the C-ABI layer where the two languages meet. It has 11 entry points in 231 lines, and its 9 raw-pointer functions contain all of the `unsafe`. The core has 0 `unsafe` sites. The cost on a real signal call is bounded at 10.91% of that call. Prior work identifies API design and the learning curve as adoption costs, but this study does not measure their size. Most of the Rust build's extra cost is building crates, not the compiler's own work. The size gap is a linkage policy. The order rests on which matched policy is chosen. Where C++ leads is the edit and compile loop. It leads only for edits that stay inside one source file.

On speed the data first splits by what is being timed. The end-to-end clock starts before a producer sends a request and stops after the worker replies. It includes the queue, wake-up, and core call. Only 0.250% of that round-trip is the core work the languages perform, so this clock cannot bound a compiler effect. A second measurement removes the queue and cross-thread handoff and times the core call alone. The core is 1.6× faster in Rust in that measurement. At the chosen load of 56 kops/s the paired test finds 1.5 $\mu$s in Rust's favour. At p99.99 the slowest Rust variant sits 22 $\mu$s below the fastest C++ one. The campaign then shows an allocator split at 448 kops/s and a 1.7× C++ lead deeper into

overload. Moving the fill after the timed window on this same host removes both large patterns and leaves a smaller Rust lead. CPU per completed call falls from 1.7× to 1.03×. The gaps do not rank the two services, and overload service latency remains unresolved. On RAM the Rust build carries a few MiB more fixed footprint under both heaps. It does so in the same direction and MiB scale on all three hosts timed. The heap choice costs about as much on both bare-metal hosts, and an order of magnitude more on the guest this study used first.

Taken as a set these support a narrow claim. It is for stateful, concurrent, partitionable control plane services of this kind. At the loads where the service still meets its sent rate, the choice is which classes of defect a team wants to make impossible to write. The other half is whether it will pay the design and training cost of getting them. Past that point the campaign includes substantial recorder work and adds no second language rank. What the thesis adds is not a verdict. It is the frozen spec, the parity tested pair, and a timing harness that reaches past the saturation region. A split of the round-trip says how much of what it times the language can own. A defect list maps where the guarantee stops.

Future work answers the gaps one at a time. The same-host control is now done. In-run and deferred fill were repeated on one host, with the pin and `nohz_full` held. The large gaps move with the fill. What remains is the smaller Rust throughput lead on the deferred plateau, which this visit does not explain. The next job needs no new host. It is to vary the transparent hugepage policy on one host with all else held fixed. All three hosts agree that the Rust build carries the larger footprint. They disagree about how that sits against the cost of the heap. The guest put it an order of magnitude higher, and neither bare-metal host does. The hugepage policy is the candidate this study can name but not test, because it was not the same on each host and only the system allocator variant gets it. Beyond that, replaying anonymised production traces would check the call mix. The harness can also take further control plane parts or the data heavy layer-2 and physical layer (PHY) path. A part with truly shared mutable state would work the two sides in a way a shared-nothing design does not. Testing Rust async runtimes against the thread per shard model would split the runtime from the language in the tail result [32]. Timing what a split of the two heaps in one process costs would price what keeping a core free of `unsafe` is worth [62, 63]. A longer case study would track defect counts and review effort while adding Rust a piece at a time. That is what would settle the human cost this study can only name.

# Statement on the Use of AI Tools

**Tool.** Anthropic's Claude Opus 5, Cursor Grok 4.6, and Cursor Grok 4.7 are the generative AI systems used in preparing this thesis. All three ran through the Cursor development environment from June 2026 through the writing of this thesis. University of Helsinki policy forbids naming a language model as an author. They are not cited as a source, which is the author's own choice.

**What the assistance covered.** The models drafted and revised the running prose, and proposed the order of the arguments. They wrote and refactored both versions of the UE Context Management service. They also wrote the load generator, the fault-injection catalogue, and the FFI prototype. The analysis behind the bootstrap intervals and the paired permutation tests is their work too, as are the figure, build and LaTeX sources. The abstract is this programme's maturity test. University policy never permits a language model in one. In this draft it has not yet been rewritten unaided, and this statement will say so until it has been.

**What the models did not settle.** The research problem, the four research questions, the three hypotheses, and the study's main control are the author's. That control is one frozen language-neutral specification and one concurrency model, so language remains the only planned gap between the two versions. The author also chose to carry defect classes from outside Rust's compile-time guarantee, which is what keeps RQ2 falsifiable. Every measurement was run by the author, on the machines each result names. The quantities the thesis quotes reach the page as macros rebuilt from the raw logs. The readings are the author's too.

**How the output was checked.** All model output, prose and code, was read by the author before it entered the thesis. Every bibliography entry was resolved against Crossref, DataCite, or the issuing body. An audit then graded how far each claim could be traced. The first grade is the page or clause itself. The second is a publisher-deposited record that carries the claim. The third is confirmed metadata behind a paywall. An entry whose supporting passage was never opened is listed, not absorbed. The audit found four passages that overstated or misplaced their source. One was a quotation absent from the work it was attributed to. All four were corrected. Code was checked against a conformance suite whose transcripts are diffed line by line. It was also checked against a fault catalogue that fixes each expected outcome before the defect is injected.

**Responsibility.** The author has reviewed the whole of this thesis and is responsible for all of it. Where it is wrong, the error is the author's.

# Bibliography


[1] 3GPP. *NG-RAN; Architecture description*. Technical Specification (TS) 38.401. V17.5.0, Release 17. 3rd Generation Partnership Project (3GPP), June 2023. URL: https://www.3gpp.org/ftp/Specs/archive/38_series/38.401/38401-h50.zip.

[2] 3GPP. *NR; NR and NG-RAN Overall description; Stage 2*. Technical Specification (TS) 38.300. V17.5.0, Release 17. 3rd Generation Partnership Project (3GPP), June 2023. URL: https://www.3gpp.org/ftp/Specs/archive/38_series/38.300/38300-h50.zip.

[3] M. Polese, L. Bonati, S. D'Oro, S. Basagni, and T. Melodia. "Understanding O-RAN: Architecture, Interfaces, Algorithms, Security, and Research Challenges". In: *IEEE Communications Surveys & Tutorials* 25.2 (2023), pp. 1376–1411. DOI: 10.1109/COMST.2023.3239220. URL: https://doi.org/10.1109/COMST.2023.3239220.

[4] 3GPP. *NR; Radio Resource Control (RRC); Protocol specification*. Technical Specification (TS) 38.331. V17.5.0, Release 17. 3rd Generation Partnership Project (3GPP), July 2023. URL: https://www.3gpp.org/ftp/Specs/archive/38_series/38.331/38331-h50.zip.

[5] 3GPP. *Study on scenarios and requirements for next generation access technologies*. Technical Report (TR) 38.913. V18.0.0, Release 18. Clause 7.4 sets the control plane latency target, for the transition from a battery-efficient state to the start of continuous data transfer, at 10 ms. 3rd Generation Partnership Project (3GPP), Apr. 2024. URL: https://www.3gpp.org/ftp/Specs/archive/38_series/38.913/38913-i00.zip.

[6] J. Dean and L. A. Barroso. "The Tail at Scale". In: *Communications of the ACM* 56.2 (2013), pp. 74–80. DOI: 10.1145/2408776.2408794. URL: https://doi.org/10.1145/2408776.2408794.

[7] L. Barroso, M. Marty, D. Patterson, and P. Ranganathan. "Attack of the Killer Microseconds". In: *Communications of the ACM* 60.4 (2017), pp. 48–54. DOI: 10.1145/3015146. URL: https://doi.org/10.1145/3015146.

[8] M. Miller. *Trends, challenge, and shifts in software vulnerability mitigation*. BlueHat IL, Microsoft Security Response Center (MSRC). Approximately 70% of Microsoft CVEs are memory-safety issues. 2019. URL: https://github.com/microsoft/MSRC-Security-Research/tree/master/presentations/2019_02_BlueHatIL.

[9] J. Vander Stoep. *Memory Safe Languages in Android 13*. Google Security Blog. Dec. 2022. URL: https://security.googleblog.com/2022/12/memory-safe-languages-in-android-13.html.

[10] S. Klabnik and C. Nichols. *The Rust Programming Language*. 2nd. San Francisco, CA: No Starch Press, 2023. ISBN: 978-1-7185-0310-6. URL: https://openlibrary.org/isbn/9781718503106.

[11] R. Jung, J.-H. Jourdan, R. Krebbers, and D. Dreyer. "RustBelt: Securing the Foundations of the Rust Programming Language". In: *Proceedings of the ACM on Programming Languages* 2.POPL (2018), 66:1–66:34. DOI: 10.1145/3158154. URL: https://doi.org/10.1145/3158154.

[12] A. Balasubramanian, M. S. Baranowski, A. Burtsev, A. Panda, Z. Rakamarić, and L. Ryzhyk. "System Programming in Rust: Beyond Safety". In: *Proceedings of the 16th Workshop on Hot Topics in Operating Systems (HotOS)*. 2017, pp. 156–161. DOI: 10.1145/3102980.3103006. URL: https://doi.org/10.1145/3102980.3103006.

[13] M. Polese, N. Mohamadi, S. D'Oro, L. Bonati, and T. Melodia. "Beyond Connectivity: An Open Architecture for AI-RAN Convergence in 6G". In: *IEEE Communications Magazine* 64.7 (2026), pp. 90–95. DOI: 10.1109/MCOM.001.2500438. URL: https://doi.org/10.1109/MCOM.001.2500438.

[14] ITU-R. *Framework and overall objectives of the future development of IMT for 2030 and beyond*. Recommendation ITU-R M.2160-0. Approved 13 November 2023. Defines the IMT-2030 ("6G") framework and its development timeline. International Telecommunication Union, Radiocommunication Sector, Nov. 2023. URL: https://www.itu.int/rec/R-REC-M.2160/en.

[15] AI-RAN Alliance. *AI-RAN Platform and Infrastructure Orchestrator*. AI-RAN Alliance, Working Group 2 (AI-and-RAN). Version 10-rc1, clause 2.2. Defines AI-and-RAN as one common IT infrastructure accommodating both AI and RAN workloads, which the infrastructure is required to run concurrently, and notes that the two kinds of workload may have different service-level agreements. Contributors from Ericsson, Nokia, Nvidia, Red Hat, Samsung, SoftBank, zTouch Networks and Northeastern University. Accessed August 2026. Feb. 2026. URL: https://ai-ran.org/documents/AI-RAN-Platform-Infrastructure-Orchestrator.pdf.

[16] J. Vander Stoep. *Rust in Android: Move Fast and Fix Things*. Google Security Blog. Nov. 2025. URL: https://blog.google/security/rust-in-android-move-fast-fix-things/.

[17] A. Rebert and C. Kern. *Secure by Design: Google's Perspective on Memory Safety*. Google Security Engineering, Technical Report. 2024. URL: https://research.google/pubs/secure-by-design-googles-perspective-on-memory-safety/.

[18] R. N. M. Watson, J. Baldwin, D. Chisnall, T. Chen, J. Clarke, B. Davis, N. W. Filardo, B. Gutstein, G. Jenkinson, B. Laurie, A. Mazzinghi, S. W. Moore, P. G. Neumann, H. Okhravi, A. Richardson, A. Rebert, P. Sewell, L. Tratt, M. Vijayaraghavan, H. Vincent, and K. Witaszczyk. "It Is Time to Standardize Principles and Practices for Software Memory Safety". In: *Communications of the ACM* 68.2 (2025), pp. 40–45. DOI: 10.1145/3708553. URL: https://doi.org/10.1145/3708553.

[19] Cybersecurity and Infrastructure Security Agency. *The Case for Memory Safe Roadmaps: Why Both C-Suite Executives and Technical Experts Need to Take Memory Safe Coding Seriously*. Cybersecurity Information Sheet. CISA, NSA, FBI, ASD's ACSC, CCCS, NCSC-UK, NCSC-NZ, CERT NZ, Dec. 2023. URL: https://www.cisa.gov/resources-tools/resources/case-memory-safe-roadmaps.

[20] Office of the National Cyber Director. *Back to the Building Blocks: A Path Toward Secure and Measurable Software*. Technical Report. Washington, DC: The White House, Office of the National Cyber Director (ONCD), Feb. 2024. URL: https://bidenwhitehouse.archives.gov/wp-content/uploads/2024/02/Final-ONCD-Technical-Report.pdf.

[21] European Parliament and Council of the European Union. *Regulation (EU) 2024/2847 on horizontal cybersecurity requirements for products with digital elements (Cyber Resilience Act)*. Official Journal of the European Union, OJ L, 2024/2847. Nov. 2024. URL: https://eur-lex.europa.eu/eli/reg/2024/2847/oj.

[22] Dell'Oro Group. *RAN Mostly Stable in 3Q 2025*. Dell'Oro Group, RAN Quarterly Report summary. The five largest RAN suppliers by worldwide revenue (Huawei, Ericsson, Nokia, ZTE, Samsung) accounted for 96% of the 1Q25–3Q25 RAN market, up from 95% in 2024. Accessed August 2026. Nov. 2025. URL: https://www.delloro.com/news/ran-mostly-stable-in-3q-2025/.

[23] Dell'Oro Group. *RAN Market Stabilized in 2025*. Dell'Oro Group, RAN Quarterly Report summary. Full-year 2025 preliminary findings: supplier rankings unchanged, and overall market concentration measured by the Herfindahl–Hirschman Index

reached a ten-year high. Accessed August 2026. Feb. 2026. URL: https://www.delloro.com/news/ran-market-stabilized-in-2025/.

[24] J. Almodovar and A. Sultan. *The SA1 road to 6G*. 3GPP Highlights, Issue 09. Written by the chair and the secretary of 3GPP WG SA1. Release 20 carries the 6G studies and Release 21 the normative work. ITU-R sets early 2029 for technology proposals and mid-2030 for the submitted specifications. Accessed August 2026. Dec. 2024. URL: https://www.3gpp.org/news-events/3gpp-news/sa1-6g-road.

[25] B. Qin, Y. Chen, Z. Yu, L. Song, and Y. Zhang. "Understanding Memory and Thread Safety Practices and Issues in Real-World Rust Programs". In: *Proceedings of the 41st ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*. 2020, pp. 763–779. DOI: 10.1145/3385412.3386036. URL: https://doi.org/10.1145/3385412.3386036.

[26] A. Meneely, A. Green, T. Jaafari, M. Fluet, and B. Keller. ""Just Use Rust": A Best-Case Historical Study of Open Source Vulnerabilities in C". In: *Proceedings of the 3rd IEEE/ACM International Workshop on Software Vulnerability Management (SVM)*. 2025, pp. 25–32. DOI: 10.1109/SVM66695.2025.00008. URL: https://doi.org/10.1109/SVM66695.2025.00008.

[27] 3GPP. *Service requirements for the 5G system; Stage 1*. Technical Specification (TS) 22.261. V18.6.0, Release 18. 3rd Generation Partnership Project (3GPP), Mar. 2022. URL: https://www.3gpp.org/ftp/Specs/archive/22_series/22.261/22261-i60.zip.

[28] 3GPP. *NG-RAN; E1 Application Protocol (E1AP)*. Technical Specification (TS) 38.463. V17.0.0, Release 17. 3rd Generation Partnership Project (3GPP), Apr. 2022. URL: https://www.3gpp.org/ftp/Specs/archive/38_series/38.463/38463-h00.zip.

[29] R. Karlbäck and A. Orö. "Holistic View on Alternative Programming Languages for Radio Access Network Applications in Cloud and Embedded Deployments". Carried out at Ericsson AB. Report LIU-IDA/LITH-EX-A–2021/023–SE. Master's thesis. Linköping University, Department of Computer and Information Science, 2021. URL: https://urn.kb.se/resolve?urn=urn:nbn:se:liu:diva-176307.

[30] V. Olsson. "Rust Programming Language as an Alternative to C for RAN Management Applications". Carried out at Ericsson AB. Master's thesis. KTH Royal Institute of Technology, School of Electrical Engineering and Computer Science, 2023. URL: https://urn.kb.se/resolve?urn=urn:nbn:se:kth:diva-325822.

[31] J. Olsson and I. Stenström. "Rust Async for Thread-per-Core Packet Processing". Carried out at Ericsson AB. Report LIU-IDA/LITH-EX-A–23/052–SE. Master's thesis. Linköping University, Department of Computer and Information Science, 2023. URL: https://urn.kb.se/resolve?urn=urn:nbn:se:liu:diva-197029.

[32] A. Arvidsson and M. Kaller. "Evaluating Asynchronous Programming Frameworks for 5G Control Plane Procedures". Carried out at Ericsson AB. Report LIU-IDA/LITH-EX-A–25/055–SE. Master's thesis. Linköping University, Department of Computer and Information Science, 2025. URL: https://urn.kb.se/resolve?urn=urn:nbn:se:liu:diva-214746.

[33] C. Emberg and A. Wadsten. "Attainable Safety and Long-Term Resource-Efficiency Using Rust: Evaluating the Viability of Rust in Terms of Development Effort and Performance for the Decompilation of ASN.1 Messages". Carried out at Ericsson, Linköping. Report LIU-IDA/LITH-EX-A–23/078–SE. Master's thesis. Linköping University, Department of Computer and Information Science, 2023. URL: https://urn.kb.se/resolve?urn=urn:nbn:se:liu:diva-195552.

[34] R. Pereira, M. Couto, F. Ribeiro, R. Rua, J. Cunha, J. P. Fernandes, and J. Saraiva. "Energy Efficiency Across Programming Languages: How Do Energy, Time, and Memory Relate?" In: *Proceedings of the 10th ACM SIGPLAN International Conference on Software Language Engineering (SLE)*. 2017, pp. 256–267. DOI: 10.1145/3136014.3136031. URL: https://doi.org/10.1145/3136014.3136031.

[35] P. Emmerich, S. Ellmann, F. Bonk, A. Egger, E. García Sánchez-Torija, T. Günzel, S. Di Luzio, A. Obada, M. Stadlmeier, S. Voit, and G. Carle. "The Case for Writing Network Drivers in High-Level Programming Languages". In: *Proceedings of the 2019 ACM/IEEE Symposium on Architectures for Networking and Communications Systems (ANCS)*. 2019, pp. 1–13. DOI: 10.1109/ANCS.2019.8901892. URL: https://doi.org/10.1109/ANCS.2019.8901892.

[36] F. Suchert and J. Castrillon. "STAMP-Rust: Language and Performance Comparison to C on Transactional Benchmarks". In: *Benchmarking, Measuring, and Optimizing (Bench 2022)*. Vol. 13852. Lecture Notes in Computer Science. Springer, 2023, pp. 160–175. DOI: 10.1007/978-3-031-31180-2_10. URL: https://doi.org/10.1007/978-3-031-31180-2_10.

[37] R. Gjomemo, P. H. Phung, E. Ballou, K. S. Namjoshi, V. N. Venkatakrishnan, and L. Zuck. "Leveraging Static Analysis Tools for Improving Usability of Memory Error Sanitization Compilers". In: *Proceedings of the 2016 IEEE International Conference on Software Quality, Reliability and Security (QRS)*. Two of the six authors are

affiliated with Nokia. 2016, pp. 323–334. DOI: 10.1109/QRS.2016.44. URL: https://doi.org/10.1109/QRS.2016.44.

[38] B. Schroeder, A. Wierman, and M. Harchol-Balter. "Open Versus Closed: A Cautionary Tale". In: *Proceedings of the 3rd USENIX Symposium on Networked Systems Design and Implementation (NSDI '06)*. 2006. URL: https://www.usenix.org/conference/nsdi-06/open-versus-closed-cautionary-tale.

[39] R. Natella, D. Cotroneo, and H. S. Madeira. "Assessing Dependability with Software Fault Injection: A Survey". In: *ACM Computing Surveys* 48.3 (2016), 44:1–44:55. DOI: 10.1145/2841425. URL: https://doi.org/10.1145/2841425.

[40] Z. Li, J. Wang, M. Sun, and J. C. S. Lui. "Detecting Cross-Language Memory Management Issues in Rust". In: *Computer Security – ESORICS 2022*. 2022, pp. 680–700. DOI: 10.1007/978-3-031-17143-7_33. URL: https://doi.org/10.1007/978-3-031-17143-7_33.

[41] B. Efron and R. J. Tibshirani. *An Introduction to the Bootstrap*. New York: Chapman & Hall, 1993. ISBN: 978-0-412-04231-7.

[42] S. Holm. "A Simple Sequentially Rejective Multiple Test Procedure". In: *Scandinavian Journal of Statistics* 6 (1979), pp. 65–70. URL: https://www.jstor.org/stable/4615733.

[43] P. Enberg, A. Rao, and S. Tarkoma. "The Impact of Thread-per-Core Architecture on Application Tail Latency". In: *Proceedings of the 2019 ACM/IEEE Symposium on Architectures for Networking and Communications Systems (ANCS)*. 2019, pp. 1–8. DOI: 10.1109/ANCS.2019.8901874. URL: https://doi.org/10.1109/ANCS.2019.8901874.

[44] G. Tene. *HdrHistogram: A High Dynamic Range Histogram*. Project documentation and reference implementation. Section "Corrected vs. raw value recording calls" documents the `recordValueWithExpectedInterval` correction applied when a response time exceeds the expected interval between requests. Accessed August 2026. URL: https://github.com/HdrHistogram/HdrHistogram.

[45] G. L. Steele Jr., D. Lea, and C. H. Flood. "Fast Splittable Pseudorandom Number Generators". In: *Proceedings of the 2014 ACM International Conference on Object Oriented Programming Systems Languages & Applications (OOPSLA)*. 2014, pp. 453–472. DOI: 10.1145/2660193.2660195. URL: https://doi.org/10.1145/2660193.2660195.

[46] L. C. Noll, K.-P. Vo, D. E. Eastlake 3rd, and T. Hansen. *The FNV Non-Cryptographic Hash Algorithm*. Request for Comments (RFC) 9923. Informational. Specifies the FNV-1a hash used inside each shard. Internet Engineering Task Force (IETF), Feb. 2026. DOI: 10.17487/RFC9923. URL: https://www.rfc-editor.org/rfc/rfc9923.

[47] T. Yin. *Lizard: An extensible cyclomatic complexity analyzer for many programming languages*. Version 1.23.0. Accessed August 2026. 2026. URL: https://github.com/terryyin/lizard.

[48] A. Georges, D. Buytaert, and L. Eeckhout. "Statistically Rigorous Java Performance Evaluation". In: *Proceedings of the 22nd ACM SIGPLAN Conference on Object-Oriented Programming Systems, Languages and Applications (OOPSLA)*. 2007, pp. 57–76. DOI: 10.1145/1297027.1297033. URL: https://doi.org/10.1145/1297027.1297033.

[49] X. Zhou, V. Leis, X. Yu, and M. Stonebraker. "OLTP Through the Looking Glass 16 Years Later: Communication Is the New Bottleneck". In: *15th Annual Conference on Innovative Data Systems Research (CIDR 2025)*. 2025. URL: https://www.vldb.org/cidrdb/papers/2025/p17-zhou.pdf.

[50] J. Hughes and L. Tratt. "Garbage Collection for Rust: The Finalizer Frontier". In: *Proceedings of the ACM on Programming Languages* 9.OOPSLA2 (2025), pp. 3588–3614. DOI: 10.1145/3763179. URL: https://doi.org/10.1145/3763179.

[51] M. Cui, H. Xu, H. Tian, and Y. Zhou. "rCanary: Detecting Memory Leaks Across Semi-Automated Memory Management Boundary in Rust". In: *IEEE Transactions on Software Engineering* 50.9 (2024), pp. 2472–2484. DOI: 10.1109/TSE.2024.3443624. URL: https://doi.org/10.1109/TSE.2024.3443624.

[52] J. Bornholt, R. Joshi, V. Astrauskas, B. Cully, B. Kragl, S. Markle, K. Sauri, D. Schleit, G. Slatton, S. Tasiran, J. Van Geffen, and A. Warfield. "Using Lightweight Formal Methods to Validate a Key-Value Storage Node in Amazon S3". In: *Proceedings of the ACM SIGOPS 28th Symposium on Operating Systems Principles (SOSP)*. 2021, pp. 836–850. DOI: 10.1145/3477132.3483540. URL: https://doi.org/10.1145/3477132.3483540.

[53] Y. Zhang, Y. Zhang, G. Portokalidis, and J. Xu. "Towards Understanding the Runtime Performance of Rust". In: *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. 2022, 140:1–140:6. DOI: 10.1145/3551349.3559494. URL: https://doi.org/10.1145/3551349.3559494.

[54] S. Harris, S. Cooksey, M. Vollmer, and M. Batty. "Rust for Morello: Always-On Memory Safety, Even in Unsafe Code (Experience Paper)". In: *Proceedings of the 37th European Conference on Object-Oriented Programming (ECOOP)*. Vol. 263. Leibniz International Proceedings in Informatics (LIPIcs). 2023, 39:1–39:27. DOI: 10.4230/LIPIcs.ECOOP.2023.39. URL: https://doi.org/10.4230/LIPIcs.ECOOP.2023.39.

[55] Y. Zhang, K. Zhang, G. Liu, Y. Cai, and S. Qin. "Two Birds One Stone: Effective Static Detection of Resource and Communication Deadlocks in Rust Programs". In: *Automated Software Engineering* 33.2, 65 (2026). Extends the preprint arXiv:2401.01114. DOI: 10.1007/s10515-026-00614-z. URL: https://doi.org/10.1007/s10515-026-00614-z.

[56] S. Lu, S. Park, E. Seo, and Y. Zhou. "Learning from Mistakes: A Comprehensive Study on Real World Concurrency Bug Characteristics". In: *Proceedings of the 13th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*. 2008, pp. 329–339. DOI: 10.1145/1346281.1346323. URL: https://doi.org/10.1145/1346281.1346323.

[57] H. Xu, Z. Chen, M. Sun, Y. Zhou, and M. R. Lyu. "Memory-Safety Challenge Considered Solved? An In-Depth Study with All Rust CVEs". In: *ACM Transactions on Software Engineering and Methodology* 31.1 (2022), 3:1–3:25. DOI: 10.1145/3466642. URL: https://doi.org/10.1145/3466642.

[58] A. N. Evans, B. Campbell, and M. L. Soffa. "Is Rust Used Safely by Software Developers?" In: *Proceedings of the 42nd International Conference on Software Engineering (ICSE)*. 2020, pp. 246–257. DOI: 10.1145/3377811.3380413. URL: https://doi.org/10.1145/3377811.3380413.

[59] A. Sundvisson and S. Hofer. "User-Space Device Drivers on Embedded Linux: A Comparative Case Study of C and Rust". Bachelor's thesis. Blekinge Institute of Technology, Department of Software Engineering, 2026. URL: https://urn.kb.se/resolve?urn=urn:nbn:se:bth-29775.

[60] K. R. Fulton, A. Chan, D. Votipka, M. Hicks, and M. L. Mazurek. "Benefits and Drawbacks of Adopting a Secure Programming Language: Rust as a Case Study". In: *Proceedings of the 17th Symposium on Usable Privacy and Security (SOUPS)*. USENIX Association, 2021, pp. 597–616. URL: https://www.usenix.org/conference/soups2021/presentation/fulton.

[61] W. Crichton and S. Krishnamurthi. “Profiling Programming Language Learning”. In: *Proceedings of the ACM on Programming Languages* 8.OOPSLA1 (2024), pp. 29–54. DOI: 10.1145/3649812. URL: https://doi.org/10.1145/3649812.

[62] E. Rivera, S. Mergendahl, H. Shrobe, H. Okhravi, and N. Burow. “Keeping Safe Rust Safe with Galeed”. In: *Proceedings of the 37th Annual Computer Security Applications Conference (ACSAC)*. 2021, pp. 824–836. DOI: 10.1145/3485832.3485903. URL: https://doi.org/10.1145/3485832.3485903.

[63] P. Kirth, M. Dickerson, S. Crane, P. Larsen, A. Dabrowski, D. Gens, Y. Na, S. Volckaert, and M. Franz. “PKRU-Safe: Automatically Locking Down the Heap Between Safe and Unsafe Languages”. In: *Proceedings of the 17th European Conference on Computer Systems (EuroSys)*. 2022, pp. 132–148. DOI: 10.1145/3492321.3519582. URL: https://doi.org/10.1145/3492321.3519582.

# Appendix A Key terms

Table A.1 collects the network, measurement and safety terms used throughout the thesis that are not acronyms, for readers outside the telecom domain. The acronyms are written out at first use and listed together in the List of acronyms on page v.

**Table A.1:** Key terms used throughout the thesis.

| Term | Meaning |
|---|---|
| Control plane | The signalling and management path, as opposed to the user-plane data path |
| User plane | The path that carries the user's data. The control plane sets it up and tears it down |
| Signalling | The messages that create, change and release a connection, as opposed to the user's data |
| Handover | Moving one connected device from one cell to another, so the call can continue |
| UE context | The stored record for one device. The shards divide these records, and a handover or a release changes one |
| Data radio bearer | A channel for that device's data, added while the device stays connected |
| Tail latency | High-percentile latency (p99, p99.9, p99.99), the worst-case-oriented timing metric |
| Far tail | The extreme end of the latency distribution, the p99.99, past the p99 many studies stop at |
| Jitter | How much latency varies from one operation to the next, as opposed to its typical value |
| Offered load | The rate the test tries to send, counted whether or not the service answers in time |
| Saturation | The offered load beyond which the service can no longer keep up and requests start queueing |
| Saturation knee | The offered load at which one variant stops keeping up. The four variants do not share one |
| Overload | Offered load past that point, where requests queue because the service cannot keep up |
| Open-loop | A load schedule whose send times are fixed in advance, so a slow response delays no later request |
| Closed-loop | A schedule that waits for each answer before the next request, so a stall is never recorded |
| Coordinated omission | The bias an open-loop schedule avoids, in which a stalled client stops issuing requests and so never records the latency of the stall |
| Blind zone | Delays too short for the end-to-end sweep to separate. A language effect inside it does not show in that sweep |
| Shard | One of several independent slices of the UE state, each owned by exactly one worker thread |

| Term | Meaning |
|---|---|
| Bare metal | The measurement machine with no hypervisor between the program and the processor |
| Allocator | The library that hands out and takes back heap memory. Both languages are measured with the system allocator and with jemalloc |
| Data race | Two threads use the same memory at once and at least one writes, with nothing to order the two uses |
| Race condition | A bug that depends on which of two steps finishes first. Safe Rust does not rule these out, and it does not rule out deadlocks |
| In-guarantee class | A defect from the set the compiler is built to reject. In safe Rust each one is a compile error |
| Boundary class | A defect safe Rust is not built to stop. These compile in safe Rust and then fail |
| Unsafe | A region the programmer marks so the compiler stops checking it. C++ marks no such region |

## Appendix B Host and campaign configuration

Table B.1 itemises the three machines the campaigns ran on: the bare-metal host of Section 5.1 that every performance and resource table of the main sweep is drawn from, the virtualised host it replaced, and the second bare-metal host at a different design that Section 5.1 reads against it. The resident-set decomposition belongs to the second bare-metal host and is reported in Section 5.3.3. The fault catalogue was compiled and run on the development machine, whose toolchain its own caption records. Of the adoption-cost axes only build time is host-sensitive, which is why that one is reported as a paired ratio while the footprint and dependency figures are deterministic. The three small paired experiments predate the move to bare metal and carry no environment record, so each supports a ratio between two variants measured alternately in one session. Section 5.1 points here for the field-by-field record of what each campaign captured. Every column reports what was measured, and the protocol they are read against is published as `bench/TESTBED.md`. Table B.2 gathers the configuration of the service, the workload and the campaign.

**What each campaign recorded.** Each of the 1120 runs of the factorial design in Table B.2 contributes one newline-delimited JSON record. Nothing is aggregated at capture time. A record carries the configuration (language, allocator, workload, shard and producer counts, target rate), the outcome (operations completed, errors, wall-clock duration, achieved throughput), the latency distribution from the HDR histogram (mean, standard deviation, the quartiles, p50, p90, p99, p99.9, p99.99 and the maximum, in nanoseconds), a shorter block for inter-operation latency variation (mean, standard deviation, p50, p99, p99.9, the maximum and a sample count), the resource counters from `getrusage` (peak resident set size, user and system CPU seconds, CPU utilisation), and the per-operation-kind counts. A `jemalloc` variant adds that allocator's own accounting. A system-allocator variant stores those fields as unavailable rather than as zero, on the same principle as the governor and turbo state. The kind counts are what makes a divergence between the languages show up as a parity failure (Chapter 4) instead of a performance difference. Each run also persists its complete HDR bucket contents to a sidecar, so the repeats can be pooled before a far-tail percentile is read (Section 3.5), and a separate file records `perf` counters at two scopes, the end-to-end path and the operation on its own.

The three small paired experiments have a different shape. For each of 15 interleaved repeats the decomposition records the batch-mean cost in both languages, once for the operation called directly and once through the threaded service. A third dataset of the

**Table B.1:** The three machines the campaigns ran on, each reported as measured. Every hardware, kernel and toolchain cell is generated from that campaign's own environment capture rather than restated from the protocol of `bench/TESTBED.md`. The operating-system row names the machine and, where a campaign ran in a container, the image's userland beside it, so the kernel row prints one value for both. The last two rows are prose, because what an instrument set is adequate for is a judgement and not a captured field. The columns are arranged so that each adjacent pair isolates one change, with the exceptions the Kernel and Toolchain rows record. Reported sweep against virtualised host: the same design and the same container image on a different machine, one `g++` release apart within GCC 15, which is what Section 5.1 reads a hardware effect from. Reported sweep against second bare-metal host: a different shard count, load grid, repeat count and compiler on comparable hardware, which is what tests whether the findings depend on those choices. Rows where all three agree describe the source or the method rather than any machine. The *Short of protocol* marks are deliberate and are argued where they appear.

| Aspect | Reported sweep | Virtualised host, same design | Second bare-metal host, different design |
|---|---|---|---|
| Host | AMD EPYC 7443P, 24 cores, 1 socket | Intel(R) Xeon(R) CPU E5-2680 v4 guest, 16 vCPUs | AMD EPYC 7443P, 24 cores, 1 socket |
| Operating system | Ubuntu 24.04.4 LTS. Image Rocky Linux 8.10 (Green Obsidian) | Rocky Linux 8.10 (Green Obsidian) | Ubuntu 24.04.4 LTS |
| Kernel | 6.17.0-23-generic | 7.0.2-1.el8.elrepo.x86_64 | 6.17.0-23-generic |
| Hypervisor | none. Governor performance, boost off, verified on the machine | kvm. Frequency and turbo host-controlled, unobservable from inside | none. Governor performance, verified on the machine |
| CPU steal | None: no hypervisor to lose a timeslice to | Not characterised. The capture's single one-second sample read 0, and no steal probe was run on this host | 0 of 60 intervals on 16 pinned cores |
| Isolation | `nohz_full` 2-17. `isolcpus` none, omitted on purpose (§5.1) | `isolcpus` none, `nohz_full` none | `isolcpus` none, `nohz_full` none. *Short of protocol* |
| Hugepages | madvise | always, and only the system-allocator variant inherits it | madvise |
| Toolchain | `g++ 15.2.1`, `rustc 1.97.1`, pinned in a container image | `g++ 15.1.1`, `rustc 1.97.1` | `g++ 13.3.0`, `rustc 1.98`, so a compiler change is tested too |
| Pinning | Cores 2–17, 7 shards, 7 producers | Cores 2–15, 7 shards, 7 producers | Cores 2–17, 8 shards, 8 producers |
| SMT | off, verified | Absent from the topology the guest is shown, and decided on the host | off, verified |
| Repeats | 20 over 14 loads, 1120 runs | 20 over 14 loads, 1120 runs | 15 over 8 loads, 480 runs. *Short of protocol* |
| Instruments | Counters, pooled HDR sidecars, jitter, allocator accounting | None available in the guest | Counters at two scopes, HDR sidecars, allocator accounting |
| Adequate for | The pre-committed campaign-clock results. Not overload service | Everything except the far tail below saturation | Confirming campaign-clock directions survive a change of design |

same shape over 20 repeats records sustained throughput through the bounded queue and completion primitives alone. The FFI comparison of Section 5.3.5 has that structure over 15 repeats. All three predate the environment record, which is why each is read as a ratio between its two variants.

Build time is the one RQ4 axis that is a timing measurement. It is recorded over 15 interleaved repeats at 14 parallel jobs on the development machine, alternating the toolchains so the ratio survives a change of machine even though the seconds do not. The static footprint and the resolved dependency graph are read from the linked binaries and the manifests.

The fault catalogue is categorical. Each of the 14 cases yields four tool logs (AddressSanitizer, ThreadSanitizer and UndefinedBehaviorSanitizer on the C++ program, and `rustc` on the safe-Rust counterpart) and one taxonomy row carrying the three sanitiser outcomes, the detection stage, the compiler result, and the expectation registered before the study. The two original boundary cases yield a second table from the model checkers, and the `unsafe` one a third from the interpreter. The table and figure in Section 5.3.4 are generated from those logs.

Every load sweep writes the environment record of Section 3.6 beside its results, including the fields a given host does not expose. The governor and turbo state are stored as unavailable rather than left out, so the limitation of Section 5.1 is a property of the dataset. The virtualised campaign shares the sweep's design and predates the later instruments: its records carry the percentiles and the resource counters and nothing else. That absence is the measurement-validity argument of Section 5.1, which is why the campaign is kept. The second bare-metal campaign carries the same record shape as the reported sweep, including the allocator accounting that lets Section 5.3.3 decompose the resident set, but runs a different shard count, load grid, repeat count and toolchain. Each campaign has its own generator and macro prefix.

The two clock diagnostics each cover the same fourteen-load, two-language, two-allocator grid. Clock B is this host. The processor is AMD EPYC 7443P, the pin is 2–17, and `nohz_full` is 2-17, matching the campaign's AMD EPYC 7443P and `2-17`. Clock A was not repeated here and remains an earlier host, so it cannot supply this campaign's percentiles. Clock B defers the fill, and its overload percentiles are schedule drift. At the top load Rust is ahead on both heaps, at $p < 0.001$ on the system allocator and $p < 0.001$ under `jemalloc`. CPU per completed operation is 1.03, against 1.7 while the fill ran inside the window. The large ratio is the recorder. The smaller throughput lead remains.

The tables that follow are the measurements the main text cites from this appendix.

The allocation profile on the second bare-metal host was taken to name the call sites behind the resident-set difference of Section 5.3.3. Two departures from the campaign explain why it cannot. The first profile ran closed loop at 455 kops/s, past the plateau bracketed at 448–512 kops/s. Repeating it at a swept load below saturation shrank the discrepancy by most of its size and did not remove it. What remains is that profiling requires an allocator built to sample every allocation and record a backtrace, so the variant under profile is not the variant the campaign measured. Each symbolised profile puts almost all of its flat bytes on a frame that allocates nothing, 98.4% on a deallocation symbol in the C++ variant and 100.0% on the allocator's own backtrace routine in the Rust one. Attributing the footprint would need a profile taken at a swept load, on a build whose peak RSS reproduces the campaign's ordering.

**Table B.2:** Configuration of the service under test, the workload driving it, and the measurement campaign. Service parameters are fixed by the frozen specification, with one exception: the shard count is set to match the cores the host offers (Section 5.1). All of them, that one included, are identical in both languages. Campaign figures are generated from the measurement logs together with the tables that report them. The representative load is fixed rather than derived, for the reason Section 5.2 gives, and the throughput plateau is the lowest of the four variants' own plateaus, which differ.

| Parameter | Value | Note |
|---|---|---|
| *Service under test, identical in both languages* | | |
| Shards, and worker threads | 7 | one thread owns one shard. Identical in both languages, which is what the fairness control requires (§3.4) |
| Queue bound per shard | 1 024 requests | bounded and blocking, so overload is observable |
| Initial map capacity | 4 096 entries | reserved so steady state does not rehash |
| Bearers per context | at most 8 | |
| Signaling payload | 64 bytes | |
| Context ceiling per shard | 200 000 | |
| Shard selection | SplitMix64, then modulo | pinned: diffuses sequential identifiers |
| Intra-shard hash | FNV-1a, 64-bit | pinned on both sides and checked bit-for-bit |
| *Workload* | | |
| Workloads | signal, mixed | mixed walks the full RAN lifecycle |
| Active UEs | 5 000 | |
| Offered-load levels | 14 | up to 784 kops/s |
| Representative load | 56 kops/s | fixed, see caption |
| Campaign-clock throughput plateau | 392 kops/s achieved | both languages track through 392 kops/s. At 448 the Rust heaps split, and deferred fill on this host removes that split (Section 5.3.2) |
| Load model | open loop, fixed intended schedule | latency measured from intended send time |
| *Campaign* | | |
| Build variants | 4 | two languages × two allocators |
| Repeats per sweep point | 20 | variants interleaved within each repeat |
| Measured runs | 1120 | |
| Latency capture | HDR histogram, [1 ns, 3.6 s], 3 s.f. | same algorithm and configuration on both sides |
| Summary statistic | median with bootstrap 95% confidence interval | intervals reported in full, not as half-widths |
| Language comparison | exact paired permutation test | 14 tests, Holm-corrected |
| Attainable $p$ floors | 0.0000019 / 0.000027 | uncorrected / after correction |

**Table B.3:** Inter-operation latency variation at a sustained offered load of 56 kops/s, in microseconds, as a median over 20 interleaved repeats with a bootstrap 95% interval. Measured within one producer and never across producers, since two producers share no schedule and a difference between them is not a variation either one experienced. Reported for completeness rather than as evidence: no research question names this metric, and it is absent from the paired family of Section 3.5.

| Variant | p50 | p99 | p99.9 | max |
|---|---|---|---|---|
| C++ (system alloc) | 1.4 | 22.3 | 43.6 | 154.0 |
| | [1.3, 1.5] | [18.6, 33.4] | [39.6, 47.9] | [148.9, 167.0] |
| C++ (jemalloc) | 1.6 | 22.5 | 43.7 | 158.7 |
| | [1.5, 1.6] | [14.3, 31.9] | [42.4, 46.6] | [148.2, 170.0] |
| Rust (system alloc) | 1.5 | 26.9 | 40.7 | 56.9 |
| | [1.3, 1.7] | [20.9, 33.7] | [38.7, 45.1] | [53.4, 107.5] |
| Rust (jemalloc) | 1.6 | 20.9 | 44.9 | 63.4 |
| | [1.6, 1.7] | [15.9, 26.1] | [42.1, 46.6] | [55.7, 108.9] |

**Table B.4:** Where the peak resident set goes on the second bare-metal host, from `jemalloc`'s own accounting in each of the 240 runs of the two `jemalloc` variants. Medians in MiB over the steady-state loads, with the paired language difference in the last column. The two rows below the rule are the argument: the allocator holds *less* in the Rust build, and the process holds more, so the difference is outside the allocator. The final row is the quantity the other rows decompose. The derived row is the median of the per-run difference and not the difference of two medians, because the two figures it subtracts come from one process at one moment, so the column does not sum exactly.

| Component | C++ | Rust | Difference |
|---|---|---|---|
| Live allocation (`allocated`) | 0.78 | 0.62 | −0.16 |
| In active pages (`active`) | 1.43 | 0.79 | −0.64 |
| Arena bookkeeping (`metadata`) | 4.19 | 6.26 | +2.07 |
| Resident in allocator mappings (`resident`) | 11.02 | 9.10 | −1.92 |
| Outside those mappings (derived) | 4.04 | 7.86 | +3.82 |
| Peak resident set (`getrusage`) | 15.08 | 16.82 | +1.74 |